COMP 441 — Software Analysis and Testing

Week 6 — AI Baseline, Coverage and Static Analysis

Group A · Track D · TOML Kit

Period	Week 6
Evidence folder	week6
Repository	github.com/the-dezeray/tomlkit
Commit(s)	8c959b5
Generated	2026-10-27
Contents	12 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 6 evidence (Group A, Track D, tomlkit @ 8c959b5) . . . 3

Reproduce . . . 3
Files . . . 3
Requirements traceability matrix (R01–R16) . . . 3
Radon cyclomatic complexity (rank C and above) . . . 4
Radon maintainability index . . . 5
Pylint output . . . 5
CrossHair symbolic-execution counterexample . . . 7
CrossHair contract . . . 7
Assertion audit summary . . . 7
AI-only coverage summary . . . 8
Maintainers' existing coverage summary . . . 8
AI test IDs used by the RTM . . . 8
Raw evidence inventory . . . 8

Source: week6\README.md

Week 6 evidence (Group A, Track D, tomlkit @ 8c959b5)

Run everything from the repo root.

Reproduce

```
python -m venv .venv && . .venv/bin/activate
pip install -e . pytest pytest-cov radon pylint ruff crosshair-tool
git submodule update --init

# AI-only baseline (expect 385 passed, 3 xfailed; 81.7% stmt / 66.3% branch)
pytest -q --cov=tomlkit --cov-branch --cov-report=html:week6/htmlcov_ai tests_ai
# Existing suite, separate run (expect 1058 passed; 89.5% / 83.1%)
pytest -q --cov=tomlkit --cov-branch tests

python week6/scripts/assert_audit.py tests_ai week6/assertion_audit.csv
python week6/scripts/gap_by_function.py week6/cov_ai.json . 25

radon cc tomlkit -s -n C --total-average
radon mi tomlkit -s
pylint tomlkit --disable=C0114,C0115,C0116
ruff check tomlkit --select ALL --statistics

cd week6/symbolic && crosshair check ch_date.py --per_condition_timeout=60
```

Files

- `tests_ai/` — AI-generated suite + `PROMPT_LOG.md` (prompts, iterations)
- `rtm/rtm_week6.csv` — traceability matrix (16 requirements); `build_rtm.py` validates test refs
- `assertion_audit.csv` — per-test assertion verdict
- `htmlcov_ai/`, `cov_*.json` — coverage
- `radon_*`, `pylint.txt` — static analysis
- `symbolic/` — CrossHair contract + counterexample for defect D1

Requirements traceability matrix (R01–R16)

Source: week6\rtm\rtm_week6.csv

Req ID	Requirement	Source	Risk priority	AI-baseline tests (tests_ai/)	Status after AI baseline	Planned tests (level, week)
R01	Parse any valid TOML 1.1 document into a TOMLDocument whose values match the spec types	README ("1.1.0-compliant"); CHANGELOG 0.15.0 #456	High	test_ai_parser.py::TestKeys; test_ai_parser.py::TestStrings; test_ai_parser.py::TestIntegers; test_ai_parser.py::TestFloats; test_ai_parser.py::TestDates; test_ai_parser.py::TestTables; test_ai_parser.py::TestArrayOfTables	Covered (unit)	Integration: run toml-test valid corpus through parse→unwrap (Wk 9); Property: Hypothesis value round-trip (Wk 7)
R02	parse() then dumps() on an unmodified document returns the identical string (comments, whitespace, order preserved)	README ("preserves all comments, indentations, whitespace and internal element ordering")	High	test_ai_api.py::TestRoundTrip; test_ai_parser.py::TestArrays::test_multiline_trailing_comma_and_comments; test_ai_parser.py::TestWhitespaceAndNewlines::test_crlf_newlines	Covered (unit, 8 fixed examples)	Property: Hypothesis parse→dumps identity on generated docs (Wk 7); BDD scenario 'Unchanged file is byte-identical' (Wk 8)
R03	Editing a parsed document changes only the edited value; untouched comments and layout are preserved	README ("accessible and editable via an intuitive API")	High	test_ai_container.py::TestEditingPreservesStyle; test_ai_items.py::TestIntegerFloat::test_integer_keeps_trivia_comment; test_ai_toml_file.py::TestTOMLFile::test_edit_and_write	Partial: Container.append/_insert_after/_replace_at largely unexecuted (see coverage gaps)	Manual unit tests from Wk 7 surviving mutants; BDD 'Bump a version keeps comments' (Wk 8)
R04	Malformed input raises a ParseError subclass that reports the line and column of the problem	Code: exceptions.py, parser.py (parse_error)	High	test_ai_parser.py::TestErrorPositions; test_ai_parser.py::TestStrings::test_control_char_rejected; test_ai_parser.py::TestKeys::test_missing_key	Partial: ~25 negative tests assert only ParseError; D3 inline duplicate key escapes ParseError	Manual: assert exact subclass + line/col per error type (decision table, Wk 7); integration with toml-test invalid corpus (Wk 9)
R05	dumps() serialises Mapping/Table/Container input, honours sort_keys, and raises TypeError for anything else	Docstring: api.dumps; CHANGELOG #482	Medium	test_ai_api.py::TestDumps; test_ai_api.py::TestEncoders::test_register_and_unregister	Covered (unit); sort_keys ignored on wrapper path not tested	Manual: wrapper + sort_keys case (Wk 7)

Req ID	Requirement	Source	Risk priority	AI-baseline tests (tests_ai/)	Status after AI baseline	Planned tests (level, week)
R06	load()/dump() work with text and binary file-like objects	Docstring: api.load, api.dump	Medium	test_ai_api.py::TestFileLikeIO	Covered (unit)	Integration: dump→file→load round trip with real file (Wk 9)
R07	TOMLFile.read/write round-trips a file and preserves LF, CRLF and mixed line endings	Code: toml_file.py	Medium	test_ai_toml_file.py::TestTOMLFile	Covered (unit, 98% stmt)	Integration test with real filesystem + edit (Wk 9)
R08	date()/time()/datetime() factories return the right item type and reject strings of another kind	Docstring: api.date ("only accepts date strings")	Medium	test_ai_api.py::TestDateTimeFactories	FAILS: D1 date() accepts datetime strings (xfail)	Fix via TDD cycle #1 (Wk 8); Hypothesis over generated datetime strings (Wk 7)
R09	item() converts supported Python types to TOML items, raises ConvertError otherwise, and consults registered custom encoders	Docstring: items.item, api.register_encoder	Medium	test_ai_items.py::TestItemConversion; test_ai_api.py::TestEncoders	Covered (unit); many checks are type-only	Property: Hypothesis item(x).unwrap() == x for generated JSON-like data (Wk 7)
R10	string() applies TOML escaping for basic strings, supports literal/multiline forms, and rejects sequences invalid for the chosen form	Docstring: api.string; TOML spec §String; CHANGELOG (leading-newline fix)	Medium	test_ai_api.py::TestStringFactory; test_ai_items.py::TestStringFromRaw; test_ai_utils.py::TestEscapeString	Covered (unit)	Property: string(s) re-parses to s for arbitrary text (Wk 7)
R11	Arithmetic on Integer/Float/Date/DateTime items returns TOML items of the same kind and keeps trivia	Code: items.py (Integer.__add__ etc.)	Low	test_ai_items.py::TestIntegerFloat; test_ai_items.py::TestDates	Covered (unit)	—
R12	Array editing (append/insert/delete/set/add_line) yields valid, re-parseable output; slice assignment is rejected	Docstring: Array.add_line; CHANGELOG #499, #527; commit #600	High	test_ai_items.py::TestArray	Partial: Array.__delitem__ and insert edge paths unexecuted	Manual BVA tests on index 0 / -1 / len; state-transition tests for comma placement (Wk 7)
R13	Out-of-order and dotted-key tables can be read, edited and deleted; re-serialisation follows the documented limitation	README §Limitations; CHANGELOG #261, #536, #545	High	test_ai_container.py::TestDottedAndOutOfOrder; test_ai_parser.py::TestTables::test_out_of_order_tables_round_trip	Partial: OutOfOrderTableProxy.__setitem__ / _merge_aot_fragment unexecuted	Integration: proxy + container + renderer scenarios (Wk 9)
R14	Comments created through the API always produce valid TOML; line breaks are rejected or split	Docstring: api.comment; CHANGELOG #449, commit #544	Medium	test_ai_api.py::TestTriviaFactories::test_multiline_comment; test_ai_items.py::TestItemDecorations::test_comment_rejects_newline; test_ai_items.py::TestArray::test_add_line_comment_with_newline_rejected	FAILS for \r: D6 comment('a\rb') writes unparseable TOML (no AI test caught it)	Manual: \r and \r\n cases (Wk 7); TDD fix (Wk 8)
R15	Keys built with key() always serialise to a valid, re-parseable key	Docstring: api.key; reverse-engineered from code	Low	test_ai_api.py::TestKeyFactory	FAILS: D5 key([]) serialises as ' = 1' (no AI test caught it)	Manual BVA: empty / one / many segments (Wk 7)
R16	Parsing time grows roughly linearly with document size (no quadratic blow-up)	CHANGELOG 0.15.1 #479, #503, #517, #525 (performance fixes); Risk R5	Medium	—	UNTESTED by AI suite	Light-touch performance test with Locust/pytest-benchmark on large lock-file style input (Wk 9)

Radon cyclomatic complexity (rank C and above)

Source: week6\radon_cc.txt

```
tomlkit/container.py
    M 225:4 Container.append - F (74)
    M 871:4 Container._replace_at - F (41)
    M 661:4 Container._render_table - E (31)
    M 1173:4 OutOfOrderTableProxy.__setitem__ - C (16)
    M 420:4 Container._validate_table_candidate - C (15)
    M 563:4 Container._insert_at - C (15)
    F 1272:0 _equal_with_nan - C (13)
    M 516:4 Container._insert_after - C (13)
    M 153:4 Container._get_last_index_before_table - C (12)
    M 633:4 Container.as_string - C (11)
tomlkit/parser.py
    M 867:4 Parser._parse_string - E (40)
    M 1006:4 Parser._parse_table - E (37)
    M 480:4 Parser._parse_value - D (25)
    M 292:4 Parser._parse_comment_trail - D (23)
    M 744:4 Parser._parse_number - D (22)
    M 633:4 Parser._parse_array - C (13)
    M 175:4 Parser.parse - C (12)
    M 811:4 Parser._parse_escaped_char - C (12)
tomlkit/_utils.py
    F 48:0 parse_rfc3339 - C (11)
tomlkit/items.py
    M 1655:4 Array.__delitem__ - E (36)
```

```
    F 114:0 item - E (31)
    M 2055:4 InlineTable.as_string - D (30)
    M 1595:4 Array.insert - D (26)
    M 1493:4 Array.add_line - D (24)
    M 2316:4 AoT.insert - C (15)
    M 1395:4 Array._group_values - C (11)

524 blocks (classes, functions, methods) analyzed.
Average complexity: A (3.0381679389312977)
```

Radon maintainability index

Source: week6\radon_mi.txt

```
tomlkit/container.py - C (0.00)
tomlkit/toml_file.py - A (74.02)
tomlkit/_compat.py - A (66.26)
tomlkit/__init__.py - A (100.00)
tomlkit/parser.py - C (0.00)
tomlkit/_types.py - A (80.64)
tomlkit/source.py - A (57.62)
tomlkit/api.py - A (64.23)
tomlkit/exceptions.py - A (68.60)
tomlkit/toml_document.py - A (100.00)
tomlkit/_utils.py - A (52.71)
tomlkit/items.py - C (0.00)
```

Pylint output

Source: week6\pylint.txt

```
************* Module tomlkit.container
tomlkit/container.py:1: [C0302(too-many-lines), ] Too many lines in module (1287/1000)
tomlkit/container.py:139: [W0212(protected-access), Container._handle_dotted_key] Access to a protected member _dotted o
f a client class
tomlkit/container.py:142: [W0212(protected-access), Container._handle_dotted_key] Access to a protected member _dotted o
f a client class
tomlkit/container.py:135: [R1711(useless-return), Container._handle_dotted_key] Useless return at end of function or met
hod
tomlkit/container.py:225: [R0914(too-many-locals), Container.append] Too many local variables (19/15)
tomlkit/container.py:274: [R1705(no-else-return), Container.append] Unnecessary "elif" after "return", replace only that
 "elif" with "if"
tomlkit/container.py:261: [R1702(too-many-nested-blocks), Container.append] Too many nested blocks (6/5)
tomlkit/container.py:304: [R1705(no-else-return), Container.append] Unnecessary "elif" after "return", replace only that
 "elif" with "if"
tomlkit/container.py:261: [R1702(too-many-nested-blocks), Container.append] Too many nested blocks (6/5)
tomlkit/container.py:389: [R1705(no-else-return), Container.append] Unnecessary "else" after "return", remove the "else"
 and de-indent the code inside it
tomlkit/container.py:400: [W0212(protected-access), Container.append] Access to a protected member _previous_item of a c
lient class
tomlkit/container.py:225: [R0911(too-many-return-statements), Container.append] Too many return statements (9/6)
tomlkit/container.py:225: [R0912(too-many-branches), Container.append] Too many branches (46/12)
tomlkit/container.py:225: [R0915(too-many-statements), Container.append] Too many statements (92/50)
tomlkit/container.py:425: [W0212(protected-access), Container._validate_table_candidate] Access to a protected member _m
ap of a client class
tomlkit/container.py:445: [W0212(protected-access), Container._validate_table_candidate] Access to a protected member _m
ap of a client class
tomlkit/container.py:451: [W0212(protected-access), Container._validate_table_candidate] Access to a protected member _m
ap of a client class
tomlkit/container.py:661: [R0912(too-many-branches), Container._render_table] Too many branches (16/12)
tomlkit/container.py:871: [R0914(too-many-locals), Container._replace_at] Too many local variables (16/15)
tomlkit/container.py:871: [R0912(too-many-branches), Container._replace_at] Too many branches (17/12)
tomlkit/container.py:871: [R0915(too-many-statements), Container._replace_at] Too many statements (53/50)
tomlkit/container.py:994: [W0613(unused-argument), Container._getstate] Unused argument 'protocol'
tomlkit/container.py:1077: [W0212(protected-access), OutOfOrderTableProxy.validate] Access to a protected member _body o
f a client class
tomlkit/container.py:1083: [W0212(protected-access), OutOfOrderTableProxy.validate] Access to a protected member _valida
te_out_of_order_table of a client class
tomlkit/container.py:1126: [W0212(protected-access), OutOfOrderTableProxy._merge_aot_fragment] Access to a protected mem
ber _map of a client class
tomlkit/container.py:1128: [W0212(protected-access), OutOfOrderTableProxy._merge_aot_fragment] Access to a protected mem
ber _map of a client class
tomlkit/container.py:1136: [W0212(protected-access), OutOfOrderTableProxy._merge_aot_fragment] Access to a protected mem
ber _body of a client class
tomlkit/container.py:1141: [W0212(protected-access), OutOfOrderTableProxy._merge_aot_fragment] Access to a protected mem
ber _body of a client class
tomlkit/container.py:1141: [W0212(protected-access), OutOfOrderTableProxy._merge_aot_fragment] Access to a protected mem
ber _body of a client class
tomlkit/container.py:1174: [C0415(import-outside-toplevel), OutOfOrderTableProxy.__setitem__] Import outside toplevel (i
tems.item)
tomlkit/container.py:1173: [R0912(too-many-branches), OutOfOrderTableProxy.__setitem__] Too many branches (14/12)
tomlkit/container.py:1227: [W0212(protected-access), OutOfOrderTableProxy._remove_table] Access to a protected member _b
```

```
ody of a client class
tomlkit/container.py:1229: [W0212(protected-access), OutOfOrderTableProxy._remove_table] Access to a protected member _r
emove_at of a client class
tomlkit/container.py:1267: [W0212(protected-access), ends_with_whitespace] Access to a protected member _previous_item o
f a client class
************* Module tomlkit.parser
tomlkit/parser.py:1: [C0302(too-many-lines), ] Too many lines in module (1297/1000)
tomlkit/parser.py:103: [W0621(redefined-outer-name), Parser.__init__] Redefining name 'string' from outer scope (line 5)
tomlkit/parser.py:146: [W0622(redefined-builtin), Parser.consume] Redefining built-in 'min'
tomlkit/parser.py:146: [W0622(redefined-builtin), Parser.consume] Redefining built-in 'max'
tomlkit/parser.py:164: [W1113(keyword-arg-before-vararg), Parser.parse_error] Keyword argument before variable positiona
l arguments list in the definition of parse_error function
tomlkit/parser.py:259: [R1705(no-else-return), Parser._parse_item] Unnecessary "elif" after "return", replace only that
"elif" with "if"
tomlkit/parser.py:308: [R1723(no-else-break), Parser._parse_comment_trail] Unnecessary "elif" after "break", replace onl
y that "elif" with "if"
tomlkit/parser.py:292: [R0912(too-many-branches), Parser._parse_comment_trail] Too many branches (17/12)
tomlkit/parser.py:377: [R1720(no-else-raise), Parser._parse_key_value] Unnecessary "else" after "raise", remove the "els
e" and de-indent the code inside it
tomlkit/parser.py:433: [R1705(no-else-return), Parser._parse_simple_key] Unnecessary "else" after "return", remove the "
else" and de-indent the code inside it
tomlkit/parser.py:453: [W0212(protected-access), Parser._parse_quoted_key] Access to a protected member _t of a client c
lass
tomlkit/parser.py:454: [W0212(protected-access), Parser._parse_quoted_key] Access to a protected member _t of a client c
lass
tomlkit/parser.py:488: [R1705(no-else-return), Parser._parse_value] Unnecessary "elif" after "return", replace only that
 "elif" with "if"
tomlkit/parser.py:480: [R0911(too-many-return-statements), Parser._parse_value] Too many return statements (12/6)
tomlkit/parser.py:480: [R0912(too-many-branches), Parser._parse_value] Too many branches (19/12)
tomlkit/parser.py:480: [R0915(too-many-statements), Parser._parse_value] Too many statements (53/50)
tomlkit/parser.py:677: [W0212(protected-access), Parser._parse_array] Access to a protected member _s of a client class
tomlkit/parser.py:744: [R0911(too-many-return-statements), Parser._parse_number] Too many return statements (8/6)
tomlkit/parser.py:920: [R0916(too-many-boolean-expressions), Parser._parse_string] Too many boolean expressions in if st
atement (10/5)
tomlkit/parser.py:920: [R1720(no-else-raise), Parser._parse_string] Unnecessary "elif" after "raise", replace only that
"elif" with "if"
tomlkit/parser.py:946: [R1705(no-else-return), Parser._parse_string] Unnecessary "else" after "return", remove the "else
" and de-indent the code inside it
tomlkit/parser.py:999: [W0212(protected-access), Parser._parse_string] Access to a protected member _idx of a client cla
ss
tomlkit/parser.py:1004: [W0212(protected-access), Parser._parse_string] Access to a protected member _idx of a client cl
ass
tomlkit/parser.py:867: [R0912(too-many-branches), Parser._parse_string] Too many branches (25/12)
tomlkit/parser.py:867: [R0915(too-many-statements), Parser._parse_string] Too many statements (64/50)
tomlkit/parser.py:1006: [R0914(too-many-locals), Parser._parse_table] Too many local variables (24/15)
tomlkit/parser.py:1033: [R1720(no-else-raise), Parser._parse_table] Unnecessary "elif" after "raise", replace only that
"elif" with "if"
tomlkit/parser.py:1061: [R1720(no-else-raise), Parser._parse_table] Unnecessary "elif" after "raise", replace only that
"elif" with "if"
tomlkit/parser.py:1131: [R1723(no-else-break), Parser._parse_table] Unnecessary "else" after "break", remove the "else"
and de-indent the code inside it
tomlkit/parser.py:1156: [W0212(protected-access), Parser._parse_table] Access to a protected member _validate_out_of_ord
er_table of a client class
tomlkit/parser.py:1124: [R1702(too-many-nested-blocks), Parser._parse_table] Too many nested blocks (6/5)
tomlkit/parser.py:1006: [R0912(too-many-branches), Parser._parse_table] Too many branches (30/12)
tomlkit/parser.py:1006: [R0915(too-many-statements), Parser._parse_table] Too many statements (81/50)
tomlkit/parser.py:1007: [W0613(unused-argument), Parser._parse_table] Unused argument 'parent'
************* Module tomlkit._types
tomlkit/_types.py:41: [R0903(too-few-public-methods), WrapperType] Too few public methods (0/2)
tomlkit/_types.py:50: [W0223(abstract-method), _CustomList] Method '__delitem__' is abstract in class 'MutableSequence'
but is not overridden in child class '_CustomList'
tomlkit/_types.py:50: [W0223(abstract-method), _CustomList] Method '__getitem__' is abstract in class 'Sequence' but is
not overridden in child class '_CustomList'
tomlkit/_types.py:50: [W0223(abstract-method), _CustomList] Method '__len__' is abstract in class 'Sized' but is not ove
rridden in child class '_CustomList'
tomlkit/_types.py:50: [W0223(abstract-method), _CustomList] Method '__setitem__' is abstract in class 'MutableSequence'
but is not overridden in child class '_CustomList'
tomlkit/_types.py:50: [W0223(abstract-method), _CustomList] Method 'insert' is abstract in class 'MutableSequence' but i
s not overridden in child class '_CustomList'
tomlkit/_types.py:62: [W0223(abstract-method), _CustomDict] Method '__delitem__' is abstract in class 'MutableMapping' b
ut is not overridden in child class '_CustomDict'
tomlkit/_types.py:62: [W0223(abstract-method), _CustomDict] Method '__getitem__' is abstract in class 'Mapping' but is n
ot overridden in child class '_CustomDict'
tomlkit/_types.py:62: [W0223(abstract-method), _CustomDict] Method '__iter__' is abstract in class 'Iterable' but is not
 overridden in child class '_CustomDict'
tomlkit/_types.py:62: [W0223(abstract-method), _CustomDict] Method '__len__' is abstract in class 'Sized' but is not ove
rridden in child class '_CustomDict'
tomlkit/_types.py:62: [W0223(abstract-method), _CustomDict] Method '__setitem__' is abstract in class 'MutableMapping' b
ut is not overridden in child class '_CustomDict'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__abs__' is abstract in class 'Complex' but is not ov
```

```
erridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__add__' is abstract in class 'Complex' but is not ov
erridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__and__' is abstract in class 'Integral' but is not o
verridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__ceil__' is abstract in class 'Real' but is not over
ridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__eq__' is abstract in class 'Complex' but is not ove
rridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__floor__' is abstract in class 'Real' but is not ove
rridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__floordiv__' is abstract in class 'Real' but is not
overridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__int__' is abstract in class 'Integral' but is not o
verridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__invert__' is abstract in class 'Integral' but is no
t overridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__le__' is abstract in class 'Real' but is not overri
dden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__lshift__' is abstract in class 'Integral' but is no
t overridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__lt__' is abstract in class 'Real' but is not overri
dden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__mod__' is abstract in class 'Real' but is not overr
idden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__mul__' is abstract in class 'Complex' but is not ov
erridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__neg__' is abstract in class 'Complex' but is not ov
erridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__or__' is abstract in class 'Integral' but is not ov
erridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__pos__' is abstract in class 'Complex' but is not ov
erridden in child class '_CustomInt'
tomlkit/_types.py:74: [W0223(abstract-method), _CustomInt] Method '__pow__' is abstract in class 'Integral' but is not o
verridden in child class '_CustomInt'
```

Showing the first 100 of 234 lines. Full file: `week6\pylint.txt`.

CrossHair symbolic-execution counterexample

Source: week6\symbolic\crosshair_output.txt

```
ch_date.py:10: error: false when calling date_never_returns_datetime('0001-10-30 00:00') (which returns False)
```

CrossHair contract

Source: week6\symbolic\ch_date.py

```
"""CrossHair contracts: api.date() should never return a datetime."""
import datetime as _dt
import re
import tomlkit
from tomlkit._utils import parse_rfc3339

def date_never_returns_datetime(raw: str) -> bool:
    """
    pre: re.fullmatch(r"[0-9]{4}-[0-9]{2}-[0-9]{2}[T ][0-9]{2}:[0-9]{2}", raw)
    post: _
    """
    try:
        d = tomlkit.date(raw)
    except ValueError:
        return True
    return not isinstance(d, _dt.datetime)
```

Assertion audit summary

Source: week6\assertion_audit.csv

Value	Count	Share
STRONG	209	68.5%
WEAK	48	15.7%
PARTIAL	47	15.4%
NO-ASSERT (smoke)	1	0.3%
Total	**305**	**100.0%**

305 rows summarised by `verdict`.

AI-only coverage summary

Source: week6\cov_ai.json

Metric	Value
Statements	2,671 / 3,268
Statement coverage	81.73%
Branches	831 / 1,254
Branch coverage	66.27%
Combined coverage	77.44%
Excluded lines	118

Maintainers' existing coverage summary

Source: week6\cov_existing.json

Metric	Value
Statements	2,924 / 3,268
Statement coverage	89.47%
Branches	1,042 / 1,254
Branch coverage	83.09%
Combined coverage	87.70%
Excluded lines	118

AI test IDs used by the RTM

Source: week6\rtm\ai_test_ids.txt

```
388 tests collected in 0.07s
test_ai_api.py::TestContainerFactories::test_aot
test_ai_api.py::TestContainerFactories::test_array_default_empty
test_ai_api.py::TestContainerFactories::test_array_from_string
test_ai_api.py::TestContainerFactories::test_array_rejects_non_array
test_ai_api.py::TestContainerFactories::test_inline_table
test_ai_api.py::TestContainerFactories::test_super_table_renders_dotted_header
test_ai_api.py::TestContainerFactories::test_table
test_ai_api.py::TestDateTimeFactories::test_date
test_ai_api.py::TestDateTimeFactories::test_date_rejects_datetime
test_ai_api.py::TestDateTimeFactories::test_datetime_rejects_time
test_ai_api.py::TestDateTimeFactories::test_datetime_with_offset
test_ai_api.py::TestDateTimeFactories::test_invalid_string
test_ai_api.py::TestDateTimeFactories::test_time
test_ai_api.py::TestDateTimeFactories::test_time_rejects_date
test_ai_api.py::TestDocument::test_document_build
test_ai_api.py::TestDocument::test_document_empty
test_ai_api.py::TestDumps::test_dumps_nested_dict_creates_table
test_ai_api.py::TestDumps::test_dumps_plain_dict
test_ai_api.py::TestDumps::test_dumps_rejects_non_mapping
test_ai_api.py::TestDumps::test_dumps_sort_keys
test_ai_api.py::TestDumps::test_dumps_sort_keys_on_document
test_ai_api.py::TestDumps::test_dumps_table_item
test_ai_api.py::TestEncoders::test_encoder_must_return_item
```

Showing the first 25 of 307 lines. Full file: `week6\rtm\ai_test_ids.txt`.

The AI-only HTML coverage report is in `week6/htmlcov_ai/`; the raw JSON is `week6/cov_ai.json`. These are evidence, not printed page-for-page.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
week6\assertion_audit.csv	25 KB	
week6\cov_ai.json	388 KB	[AI-only]
week6\cov_existing.json	388 KB	
week6\pylint.txt	29 KB	
week6\radon_cc.json	146 KB	
week6\radon_cc.txt	1 KB	

File	Size	Tag
week6\radon_mi.txt	387 B	
week6\README.md	1 KB	
week6\rtm\ai_test_ids.txt	17 KB	
week6\rtm\build_rtm.py	7 KB	
week6\rtm\rtm_week6.csv	6 KB	
week6\scripts\assert_audit.py	3 KB	
week6\scripts\gap_by_function.py	1 KB	
week6\symbolic\ch_date.py	459 B	
week6\symbolic\crosshair_output.txt	112 B	

COMP 441 — Software Analysis and Testing

Week 7 — Mutation Testing v1 and Risk-Based Prioritisation

Group A · Track D · TOML Kit

Period	Week 7
Evidence folder	week7
Repository	github.com/the-dezeray/tomlkit
Commit(s)	8c959b5
Generated	2026-10-27
Contents	9 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 7 evidence: Mutation Testing Report v1 (Group A, Track D, tomlkit @ 8c959b5) . . . 3

Results (AI-only suite tests_ai/, mutmut 3.8.0) . . . 3
Reproduce . . . 3
Survivor root-cause analysis . . . 3
Test-priority rationale . . . 4
Files . . . 4
Mutation v1 summary by module . . . 4
Detection curve (risk-based order) . . . 4
Prioritisation evaluation (APFD) . . . 5
Risk-based test order (first 20 tests) . . . 5
Survivor categories . . . 6
Pytest collection order (first 20 tests) . . . 6
AI test durations (first 20 lines) . . . 6
Raw evidence inventory . . . 7

Source: week7\README.md

Week 7 evidence: Mutation Testing Report v1 (Group A, Track D, tomlkit @ 8c959b5)

Results (AI-only suite `tests_ai/`, mutmut 3.8.0)

	Mutants
Killed	3,087
Timeout (counted as detected)	26
Survived	1,890
No tests (code never executed)	386
Total	**5,389**

Mutation score = (killed + timeout) / total = **57.8%** (62.2% of mutants in executed code). The AI-only suite and manual survivor tests pass together: **404 passed, 3 xfailed**.

Reproduce

```
. .venv/bin/activate && pip install mutmut==3.8.0 hypothesis
mutmut run --max-children 8           # config in pyproject.toml [tool.mutmut]; ~20 min on WSL/Python 3.14
mutmut results --all true > week7/mutmut_results_v1.txt
mutmut show <mutant-name>             # diff of one mutant

# kill tests written for 11 survivors (each fails against its mutant):
cp -r tests_manual mutants/ && cd mutants
MUTANT_UNDER_TEST="tomlkit.items.x‖Array‖insert__mutmut_14" python -m pytest tests_manual -k test_k6_

# risk-based prioritisation + APFD evaluation
pytest -q --cov=tomlkit --cov-context=test --cov-report= tests_ai --durations=0 > week7/durations_ai.txt
pytest --collect-only -q tests_ai | grep :: > week7/collection_order.txt
python week7/scripts/kill_matrix.py mutants week7/mutmut_results_v1.txt 300 week7/kill_matrix.json
python week7/scripts/prioritise.py .coverage week7/radon_cc.json week7/churn_2y.tsv \
       week7/durations_ai.txt week7/kill_matrix.json week7
python week7/scripts/classify_survivors.py <survivor-list> <diff-dir> week7/survivor_categories_v1.csv
```

Survivor root-cause analysis

These eight cases show why the AI-only tests missed representative surviving mutants and how the manual tests target the gaps. The tests are in `tests_manual/test_kill_v1.py`.

Mutant	Why the AI-only tests missed it	Manual test and design technique
`InvalidNumberError.__init__` mutmut 3	Checked the exception type and position, but not the exact, case-sensitive message.	`test_k1_invalid_number_message_and_position_exact` checks the complete message and location (equivalence partitioning).
`Source._to_linecol` mutmut 7	Did not exercise the exact index at a newline boundary.	`test_k2_line_col_at_line_boundary` checks the character before and at the newline (boundary value analysis).
`Parser._parse_escaped_char` mutmut 1	Covered an invalid letter escape, not whitespace after a backslash in a single-line string.	`test_k3_backslash_space_rejected_in_single_line_string` checks rejection, alongside a multiline continuation control (equivalence partitioning).
`parse_rfc3339` mutmut 66	Used timezone offsets below ten hours, missing the two-digit-hour boundary.	`test_k4_two_digit_utc_offsets` checks `+10:00`, `-11:30`, and `+23:59` (boundary value analysis).
`parse_rfc3339` mutmut 163	Checked fractional-second truncation for datetimes, but not the separate time-only parsing branch.	`test_k5_time_fraction_truncated_to_microseconds` checks both time-only parsing paths (equivalence partitioning).
`Array.insert` mutmut 14	Tested positive and negative insertion positions, but not index zero.	`test_k6_insert_at_index_zero` checks the boundary position, resulting values, and round-trip serialization (boundary value analysis).
`Array.__delitem__` mutmut 13	Did not cover stepped slices, a distinct slice behavior from contiguous deletion.	`test_k7_delete_stepped_slice` checks both even- and odd-index deletion (equivalence partitioning).
`Array.add_line` mutmut 8	Tested LF, but not CR or CRLF line breaks.	`test_k8_add_line_comment_rejects_any_line_break` checks all three line-ending forms (equivalence partitioning).

The extra strict xfail in the index-zero insertion tests records a separate spacing defect in the unmutated library; it is not counted as a mutation kill. All 11 targeted manual tests were also run against their named mutants; each mutant was killed.

Test-priority rationale

The risk-based order ranks tests by the additional complexity-weighted, two-year-churn risk they cover; shorter runtime breaks ties. Mutation outcomes are excluded when constructing the order, keeping the APFD comparison out-of-sample. The top-ranked parse test covers the most new risk, especially `Container.append` (complexity 74); rank 2 targets `Container._replace_at` (41), and rank 4 exercises `InlineTable.as_string` (30). This puts high-risk container and formatting behavior early without tuning the ranking to the sampled mutants.

The 300-mutant kill-matrix sample had one mutant that did not remain killed in the fresh v1 run; it was excluded. Of the remaining sample, 258 mutants had matching test identifiers for APFD. The risk-based order achieved APFD **0.7722**, compared with **0.6369** for collection order and **0.6911** for the mean of 200 shuffled orders. It detected half of those mutants within 16 tests, versus 51 in collection order. The full ranking and evaluation are in `priority_order.csv` and `prioritisation_eval.json`.

Files

- `mutmut_results_v1.txt`, `mutation_v1_summary.json` - every mutant's status; score per module
- `survivor_categories_v1.csv` - all 1,890 survivors with diff and heuristic category
- `kill_matrix.json` - which AI tests fail for a seeded sample of 300 mutants; one stale mutant is excluded from APFD
- `priority_order.csv` - ranked execution order for the suite
- `prioritisation_eval.json`, `detection_curve.csv` - APFD of risk-based vs pytest order vs random
- `churn_2y.tsv` - lines changed per file since 2024-10-18 (git log --numstat)

Mutation v1 summary by module

Source: week7\mutation_v1_summary.json

Module	Total	Killed	Timeout	Survived	No tests	Score	Covered
TOTAL	5,389	3,087	26	1,890	386	57.8%	62.2%
api	149	118	7	24	0	83.9%	83.9%
container	1,261	595	0	597	69	47.2%	49.9%
exceptions	174	58	0	94	22	33.3%	38.2%
items	2,000	1,082	1	653	264	54.1%	62.4%
parser	1,387	876	11	469	31	64.0%	65.4%
source	127	103	2	22	0	82.7%	82.7%
toml_file	73	53	0	20	0	72.6%	72.6%
_utils	218	202	5	11	0	95.0%	95.0%

Detection curve (risk-based order)

Source: week7\detection_curve.csv

order	tests_run	pct_mutants_detected
Risk-based	1	14.7
Risk-based	5	25.6
Risk-based	10	43.0
Risk-based	20	54.7
Risk-based	30	65.9
Risk-based	40	70.5
Risk-based	50	72.1
Risk-based	75	82.2
Risk-based	100	89.5
Risk-based	150	100.0
Risk-based	200	100.0
Risk-based	250	100.0
Risk-based	300	100.0
Risk-based	149	100.0
Default order	1	14.7
Default order	5	17.8

order	tests_run	pct_mutants_detected
Default order	10	27.5
Default order	20	31.0
Default order	30	40.7
Default order	40	45.3
Default order	50	49.2
Default order	75	67.1
Default order	100	82.9
Default order	150	100.0
Default order	200	100.0
Default order	250	100.0
Default order	300	100.0
Default order	149	100.0

Prioritisation evaluation (APFD)

Source: week7\prioritisation_eval.json

Key	Value
tests	149
mutants_in_matrix	258
apfd_risk_based	0.7722
apfd_default_order	0.6369
apfd_random_mean	0.6911
apfd_random_min	0.6272
apfd_random_max	0.7532
tests_to_detect_50pct_risk_based	16
tests_to_detect_90pct_risk_based	106
tests_to_detect_50pct_default_order	51
tests_to_detect_90pct_default_order	120

Risk-based test order (first 20 tests)

Source: week7\priority_order.csv

rank	test	new_risk_covered	functions_executed	top_function
1	tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_returns_document	2417.2	47	Container.append (CC 74)
2	tests_ai/test_ai_container.py::TestEditingPreservesStyle::test_update_value_keeps_comment	552.4	5	Container._replace_at (CC 41)
3	tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[[[aot]]\nx = 1\n\n[[aot]]\nx = 2\n]	552.1	14	Container.append (CC 74)
4	tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[inline = { a = 1, b = "two" }\n]	377.1	8	InlineTable.as_string (CC 30)
5	tests_ai/test_ai_items.py::TestArray::test_delitem_last	246.6	1	Array.__delitem__ (CC 36)
6	tests_ai/test_ai_items.py::TestArray::test_add_line_with_comment	239.8	3	Array.add_line (CC 24)
7	tests_ai/test_ai_container.py::TestDottedAndOutOfOrder::test_out_of_order_add_key	203.1	4	Container.append (CC 74)
8	tests_ai/test_ai_api.py::TestEncoders::test_register_and_unregister	187.5	4	item (CC 31)
9	tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_identical	180.1	8	Container._render_table (CC 31)
10	tests_ai/test_ai_container.py::TestEditingPreservesStyle::test_add_scalar_goes_before_tables	176.9	3	Container.append (CC 74)
11	tests_ai/test_ai_api.py::TestParseAndLoads::test_loads_is_alias_of_parse	144.1	5	_equal_with_nan (CC 13)
12	tests_ai/test_ai_container.py::TestDottedAndOutOfOrder::test_dotted_keys_access	137.6	9	Container.append (CC 74)
13	tests_ai/test_ai_parser.py::TestStrings::test_string_values[a = "\\u00e9\\U0001F600"-\xe9\U0001f600]	135.9	4	Parser._parse_escaped_char (CC 12)
14	tests_ai/test_ai_items.py::TestIntegerFloat::test_integer_arithmetic_returns_integer	102.8	7	Integer._new (CC 3)

rank	test	new_risk_covered	functions_executed	top_function
15	tests_ai/test_ai_container.py::TestDottedAndOutOfOrder::test_out_of_order_delete	98.3	3	OutOfOrderTableProxy.__delitem__ (CC 7)
16	tests_ai/test_ai_items.py::TestIntegerFloat::test_integer_bitwise	68.5	5	Integer.__rshift__ (CC 2)
17	tests_ai/test_ai_api.py::TestKeyFactory::test_keys_in_document	67.6	5	Container.append (CC 74)
18	tests_ai/test_ai_items.py::TestIntegerFloat::test_float_arithmetic	54.8	4	Float.__mul__ (CC 2)
19	tests_ai/test_ai_api.py::TestKeyFactory::test_dotted_key	53.0	2	DottedKey.__init__ (CC 5)
20	tests_ai/test_ai_container.py::TestDottedAndOutOfOrder::test_out_of_order_table_proxy	52.7	6	Container.append (CC 74)

Showing the first 20 of 149 rows. Full file: `week7\priority_order.csv`.

Survivor categories

Source: week7\survivor_categories_v1.csv

Value	Count	Share
Argument dropped or set to None	756	40.0%
Boolean logic (and/or, True/False, not)	271	14.3%
Exception message or construction	256	13.5%
String literal changed (XX..XX / case)	196	10.4%
Arithmetic / assignment operator	147	7.8%
Boundary / off-by-one (< vs <=, +1, index)	122	6.5%
Control flow (continue/break/return)	96	5.1%
Other	46	2.4%
Total	**1890**	**100.0%**

1890 rows summarised by `category`.

Pytest collection order (first 20 tests)

Source: week7\collection_order.txt

```
tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_returns_document
tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_values
tests_ai/test_ai_api.py::TestParseAndLoads::test_loads_is_alias_of_parse
tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_accepts_bytes
tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_empty_string
tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_invalid_raises
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_identical
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[a = 1\n]
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[a = 1 # comment\n]
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[[t]\n  x = 'y'\n]
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[arr = [\n  1,\n  2,\n]\n]
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[inline = { a = 1, b = "two" }\n]
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[[[aot]]\nx = 1\n\n[[aot]]\nx = 2\n]
tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[\n\n# only comments\n\n]
tests_ai/test_ai_api.py::TestDumps::test_dumps_plain_dict
tests_ai/test_ai_api.py::TestDumps::test_dumps_nested_dict_creates_table
tests_ai/test_ai_api.py::TestDumps::test_dumps_sort_keys
tests_ai/test_ai_api.py::TestDumps::test_dumps_sort_keys_on_document
tests_ai/test_ai_api.py::TestDumps::test_dumps_rejects_non_mapping
tests_ai/test_ai_api.py::TestDumps::test_dumps_table_item
```

Showing the first 20 of 388 lines. Full file: `week7\collection_order.txt`.

AI test durations (first 20 lines)

Source: week7\durations_ai.txt

```
.....................................................................  [ 18%]
.....................................................................  [ 37%]
.....................................................................  [ 55%]
x....................................................................  [ 74%]
....................x................................................  [ 92%]
........................                                               [100%]
============================== warnings summary ==============================
tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_returns_document
  /mnt/c/Users/macha/Documents/comp_441/tomlkit/.venv/lib/python3.14/site-packages/coverage/control.py:803: CoverageWarn
  ing: Dynamic contexts aren't supported with core=sysmon; context data may be incomplete (no-sysmon-context); see https:/
  /coverage.readthedocs.io/en/7.16.2/messages.html#warning-no-sysmon-context
    self._warn(
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================== slowest durations ===============================
0.14s setup    tests_ai/test_ai_api.py::TestParseAndLoads::test_parse_returns_document
0.03s call     tests_ai/test_ai_items.py::TestPickle::test_bool_equals_bool
0.03s teardown tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[[t]\n  x = 'y'\n]
0.02s teardown tests_ai/test_ai_items.py::TestArray::test_clear
0.02s teardown tests_ai/test_ai_container.py::TestEditingPreservesStyle::test_update_value_keeps_comment
0.02s teardown tests_ai/test_ai_api.py::TestRoundTrip::test_round_trip_preserves_formatting[a = 1\n]
0.02s teardown tests_ai/test_ai_parser.py::TestInlineTables::test_duplicate_key_in_inline
```

Showing the first 20 of 194 lines. Full file: `week7\durations_ai.txt`.

The full 5,389-line mutmut output is `week7/mutmut_results_v1.txt`; the 300-mutant kill matrix is `week7/kill_matrix.json`. The original Week 7 PDF report is `week7/mutation-testing-report-v1.pdf`; it is kept as a raw artifact and is not appended here because it duplicates this README and uses legacy font encoding.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
week7\churn_2y.tsv	279 B	
week7\collection_order.txt	26 KB	
week7\detection_curve.csv	658 B	
week7\durations_ai.txt	16 KB	
week7\kill_matrix.json	325 KB	
week7\mutation-testing-report-v1.pdf	126 KB	[v1 baseline]
week7\mutation_v1_summary.json	1 KB	[v1 baseline]
week7\mutmut_results_v1.txt	332 KB	[v1 baseline]
week7\prioritisation_eval.json	383 B	
week7\priority_order.csv	15 KB	
week7\radon_cc.json	146 KB	
week7\README.md	6 KB	
week7\scripts\classify_survivors.py	2 KB	
week7\scripts\kill_matrix.py	1 KB	
week7\scripts\prioritise.py	5 KB	
week7\survivor_categories_v1.csv	314 KB	[v1 baseline]

COMP 441 — Software Analysis and Testing

Week 8 — TDD Cycles and BDD Acceptance Scenarios

Group A · Track D · TOML Kit

Period	Week 8
Evidence folder	week8
Repository	github.com/the-dezeray/tomlkit
Commit(s)	a4f54db / a38a3d8 / c829a38
Generated	2026-10-27
Contents	13 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

TDD log (Week 8, branch week8-tdd) . . . 3

C1 red — date() accepts datetime strings . . . 3

C1 green . . . 3

C1 refactor . . . 3

C2 red — bare CR in multiline strings . . . 3

C2 green . . . 3

C2 refactor . . . 3

C3 red — comment() with a bare CR . . . 4

C3 green . . . 4

C3 refactor . . . 4

BDD feature — edit config and preserve layout . . . 4

BDD feature — backlog scenarios . . . 4

Raw evidence inventory . . . 5

Source: week8\TDD_LOG.md

TDD log (Week 8, branch `week8-tdd`)

Each cycle has three commits: **red** (failing test only), **green** (minimal code change), **refactor** (no behaviour change, full suite green). Raw pytest output for each step is in `tdd_runs/`. Times are commit times (SAST, UTC+2).

Cycle	Defect	Step	Commit	Time	Evidence
C1	D1 `date()` accepts datetime strings	Red	`9472f8a`	2026-10-18 23:00:55	`tdd_runs/c1_red.txt`: 4 failed, 1 passed
		Green	`a4f54db`	23:01:24	one-line change in `api.date`; D1 xfail markers removed; 1,475 passed
		Refactor	`fad6c12`	23:01:45	`_parse_rfc3339_as()` shared by date/time/datetime; 1,475 passed
C2	D7 multiline strings emit a bare CR	Red	`51c6863`	23:01:59	`tdd_runs/c2_red.txt`: 6 failed, 2 passed
		Green	`a38a3d8`	23:03:22	`String.from_raw` escapes (MLB) or rejects (MLL) a lone CR; 1,485 passed
		Refactor	`26421d6`	23:03:40	regex named once as `_BARE_CR`; 1,485 passed
C3	D6 `comment()` with a bare CR	Red	`3c33afc`	23:03:51	`tdd_runs/c3_red.txt`: 2 failed, 2 passed
		Green	`c829a38`	23:04:11	`api.comment` treats a lone CR as a line break; 1,489 passed
		Refactor	`b60559f`	23:04:37	`BARE_CR` moved to `_utils` and shared by C2 and C3; 1,489 passed

Tests: `tests_tdd/test_tdd_cycles.py`. Full-suite count = tests/ + tests_ai/ + tests_manual/ + tests_property/ + tests_tdd/.

C1 red — date() accepts datetime strings

Source: week8\tdd_runs\c1_red.txt

```
=========================== short test summary info ============================
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27T07:32:00]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27 07:32:00Z]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[0001-10-30 00:00]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27T07:32:00-08:00]
4 failed, 1 passed in 0.20s
```

C1 green

Source: week8\tdd_runs\c1_green.txt

```
5 passed in 0.23s
1475 passed, 5 xfailed in 7.62s
```

C1 refactor

Source: week8\tdd_runs\c1_refactor.txt

```
1475 passed, 5 xfailed in 7.57s
```

C2 red — bare CR in multiline strings

Source: week8\tdd_runs\c2_red.txt

```
=========================== short test summary info ============================
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[a\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[\r]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[a\r\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[end\r]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_literal_rejects_bare_cr[a\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_literal_rejects_bare_cr[\r]
6 failed, 2 passed, 5 deselected in 0.19s
```

C2 green

Source: week8\tdd_runs\c2_green.txt

```
13 passed in 0.29s
1485 passed, 3 xfailed in 6.79s
```

C2 refactor

Source: week8\tdd_runs\c2_refactor.txt

```
1485 passed, 3 xfailed in 6.79s
```

C3 red — comment() with a bare CR

Source: week8\tdd_runs\c3_red.txt

```
========================== short test summary info ===========================
FAILED tests_tdd/test_tdd_cycles.py::test_c3_comment_splits_every_line_break[a\rb-# a\n# b]
FAILED tests_tdd/test_tdd_cycles.py::test_c3_comment_splits_every_line_break[a\r\rb-# a\n#\n# b]
2 failed, 2 passed, 13 deselected in 0.17s
```

C3 green

Source: week8\tdd_runs\c3_green.txt

```
17 passed in 0.17s
1489 passed, 3 xfailed in 6.63s
```

C3 refactor

Source: week8\tdd_runs\c3_refactor.txt

```
1489 passed, 3 xfailed in 6.90s
```

BDD feature — edit config and preserve layout

Source: tests_bdd\features\edit_config.feature

```
Feature: Edit a project config file without losing its formatting
  As a Python developer who keeps tool settings in pyproject.toml
  I want to change one value through tomlkit
  So that my comments, spacing and key order survive the edit

  # Level: acceptance (end to end: real file on disk -> TOMLFile -> edit -> file on disk)
  # RTM: R03 (edits preserve untouched layout), R07 (file round-trip)
  Scenario: Bump the version and keep every comment and blank line
    Given a config file containing:
      """
      # Project settings
      [project]
      name = "demo"  # do not rename
      version = "1.4.2"

      # Tools
      [tool.pytest]
      addopts = "-q"
      """
    When I open the file with tomlkit and set "project.version" to "1.5.0"
    And I save the file
    Then the file on disk is exactly:
      """
      # Project settings
      [project]
      name = "demo"  # do not rename
      version = "1.5.0"

      # Tools
      [tool.pytest]
      addopts = "-q"
      """
    And reading "project.version" from the saved file gives "1.5.0"
```

BDD feature — backlog scenarios

Source: tests_bdd\features\backlog.feature

```
# Drafted in Week 8, to be automated in Week 9 (not yet bound to step definitions).
Feature: Work with real-world TOML files safely

  # Level: acceptance
  # RTM: R04 (malformed input reports line and column)
  Scenario: A typo in a config file is reported at the right place
    Given a config file whose third line is "version = 1.4.2"
    When I try to open it with tomlkit
    Then I get a parse error that names line 3
    And the file on disk is unchanged

  # Level: acceptance
  # RTM: R07 (TOMLFile preserves Windows line endings)
  Scenario: Editing a Windows file keeps CRLF line endings
    Given a config file saved with CRLF line endings
    When I add the key "tool.ruff.line-length" with value 100 and save
    Then every line in the file ends with CRLF
    And the new key reads back as 100
```

The BDD step implementations are `tests_bdd/test_edit_config.py` and `tests_bdd/test_backlog.py`. The full red/green/refactor logs are in `week8/tdd_runs/`.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
week8\TDD_LOG.md	1 KB	
week8\tdd_runs\c1_green.txt	52 B	
week8\tdd_runs\c1_red.txt	503 B	
week8\tdd_runs\c1_refactor.txt	33 B	
week8\tdd_runs\c2_green.txt	53 B	
week8\tdd_runs\c2_red.txt	632 B	
week8\tdd_runs\c2_refactor.txt	33 B	
week8\tdd_runs\c3_green.txt	53 B	
week8\tdd_runs\c3_red.txt	317 B	
week8\tdd_runs\c3_refactor.txt	33 B	

COMP 441 — Software Analysis and Testing

Week 9 — Performance, Usability and Exploratory Testing

Group A · Track D · TOML Kit

Period	Week 9
Evidence folder	docs/week9
Repository	github.com/the-dezeray/tomlkit
Commit(s)	see source report
Generated	2026-10-27
Contents	10 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 9 Light-Touch Performance and Usability Report . . . 3

1. Performance Check . . . 3
2. Usability Check . . . 3
3. Overall Conclusion . . . 5
Evidence Files . . . 5

Week 9 Exploratory Session Report . . . 5

Session Information . . . 5
Charter . . . 5
Initial Risks . . . 5
Session Log . . . 5
Session Conclusions . . . 7
AI Co-Pilot Evaluation . . . 7
Follow-Up Recommendations . . . 7
Session End . . . 7
Performance environment . . . 7
Performance results . . . 7
Usability check — error diagnosis . . . 8
Usability check — insertion at index 0 . . . 8
Usability check — modification . . . 8
Usability check — help/documentation . . . 9
Usability check — API misuse . . . 9
Performance harness . . . 9
Raw evidence inventory . . . 10

Source: docs\week9\performance_usability_report.md

Week 9 Light-Touch Performance and Usability Report

1. Performance Check

Objective

Measure a representative tomlkit workflow consisting of parsing, modifying, serializing, and reparsing a TOML document while verifying round-trip correctness.

Environment

- Timestamp: 2026-10-25 13:18:07 CAT
- Platform: Darwin 25.6.0 arm64
- Processor: Apple M2
- Logical CPUs: 8
- Python: 3.14.7
- tomlkit: 0.15.1

Method

- Operations per repeat: 5,000
- Repeats: 5
- Total operations: 25,000
- Each operation parsed a TOML document, changed a scalar value, appended an array value, serialized the document, reparsed the output, and asserted that the changed value survived the round trip.

Results

- Repeat 1: 2.130827 seconds, 2,346.51 operations/second
- Repeat 2: 2.122317 seconds, 2,355.92 operations/second
- Repeat 3: 2.075538 seconds, 2,409.01 operations/second
- Repeat 4: 2.076870 seconds, 2,407.47 operations/second
- Repeat 5: 2.072806 seconds, 2,412.19 operations/second
- Total measured time: 10.478358 seconds
- Mean repeat time: 2.095672 seconds
- Median repeat time: 2.076870 seconds
- Minimum repeat time: 2.072806 seconds
- Maximum repeat time: 2.130827 seconds
- Overall throughput: 2,385.87 operations/second
- Correctness check: all round-trip assertions passed

Interpretation and Limitations

The benchmark provides a light-touch performance baseline for one representative workload on the recorded machine. The five repeat results were similar, and every correctness assertion passed. The result is not a formal performance guarantee because it used one machine, one document shape, one process, and a short synthetic workload.

2. Usability Check

The usability observations and top three issues will be recorded after completing the developer-facing heuristic evaluation.

Evaluation Scope

A developer-facing heuristic evaluation was performed using common parsing, error-handling, array-modification, and help-discovery tasks. The evaluation considered Nielsen principles that are applicable to a Python library API, particularly consistency and standards, error prevention, visibility of outcomes, and help and documentation.

Positive Finding: Error Diagnosis

Four invalid TOML inputs were tested: a leading-zero integer, an unterminated string, an invalid escape, and a duplicate key. Each input produced a specific exception with a descriptive message and line and column information. This supports the heuristic of helping users recognize, diagnose, and recover from errors.

Top Three Usability Issues

Issue 1: Beginning insertion produces inconsistent array spacing

Heuristic: Consistency and standards.

For the conventionally formatted array `[1, 2]`, inserting `3` at position 0 produced `[3,1, 2]`. In comparison:

- Appending rendered `[1, 2, 3]`.
- Middle insertion rendered `[1, 3, 2]`.
- Replacing the first item rendered `[3, 2]`.
- Deleting the first item rendered `[2]`.

All outputs reparsed correctly, so the issue affects formatting consistency rather than semantic correctness.

Impact: Users relying on tomlkit to preserve human-readable formatting can receive inconsistent output after an otherwise normal array modification. Repeated beginning insertions can extend the inconsistent no-space section.

Severity: Medium.

Recommendation: Preserve the existing separator style when inserting before the first item. Add a regression test requiring insertion into `[1, 2]` to render as `[3, 1, 2]`.

Issue 2: Array insertion index clamping is not explained by built-in help

Heuristic: Help and documentation; match between the system and the real world.

The tested behavior was:

- Position `-99` inserted at the beginning.
- Position `-1` inserted before the final item.
- Position `99` inserted at the end.
- Every call returned `None`.

This is consistent with Python-style sequence insertion and is not classified as a functional defect. However, the built-in docstring only states `insert value before index` and does not explain how indexes outside the array bounds are handled.

Impact: A developer unfamiliar with Python list insertion rules may expect an out-of-range error or may be uncertain where the item was inserted.

Severity: Low.

Recommendation: Expand the `Array.insert()` documentation to state that indexes below the valid range are clamped to the beginning and indexes above the valid range are clamped to the end.

Issue 3: Common modification methods provide limited formatting guidance

Heuristic: Help and documentation.

The built-in help for `Array.insert()` and `Array.append()` consists of short sequence-style descriptions. The help does not explain whether these operations preserve existing whitespace, comments, commas, or multiline layout. In contrast, `Array.add_line()` has a longer docstring with parameters and an example showing exact rendered output.

Impact: Developers must experiment or inspect output to understand the formatting consequences of common modifications, even though formatting preservation is a central use case for tomlkit.

Severity: Low.

Recommendation: Add short formatting examples to the documentation for `insert()`, `append()`, replacement, and deletion. Clarify which existing formatting elements are expected to be preserved.

Usability Conclusion

The tested parser error messages were specific and actionable. Common array modifications also preserved semantic values and generated reparsable TOML. The principal usability weakness was inconsistent spacing after insertion at the beginning of a conventionally spaced array. Two lower-severity documentation issues concerned undisclosed index clamping and limited guidance about formatting preservation.

3. Overall Conclusion

The light-touch benchmark completed 25,000 parse-modify-serialize-reparse operations in 10.478358 seconds, with an overall throughput of 2,385.87 operations per second on the recorded Apple M2 environment. All round-trip correctness assertions passed.

The usability evaluation identified one medium-severity formatting-consistency issue and two low-severity documentation issues. Error diagnosis was a strength because the tested invalid inputs produced descriptive exceptions with line and column information.

Evidence Files

- `performance_environment.txt`
- `performance_check.py`
- `performance_results.txt`
- `usability_error_check.txt`
- `usability_api_misuse_check.txt`
- `usability_insert_check.txt`
- `usability_modification_check.txt`
- `usability_help_check.txt`

Source: docs\week9\exploratory_session_report.md

Week 9 Exploratory Session Report

Session Information

- Start: 2026-10-25 13:12:42 CAT
- Project: tomlkit
- Environment: Python 3.14.7
- Testing approach: Exploratory and ad-hoc testing with an AI co-pilot

Charter

Explore how tomlkit preserves array formatting during insertion, deletion, serialization, and reparsing, with particular attention to boundary indexes, unusual spacing, comments, and the known D8 spacing defect.

Initial Risks

- Array modifications may preserve values but damage user formatting.
- Boundary indexes may behave differently from middle indexes.
- Serialized output may remain valid TOML while losing expected spacing or comments.
- Repeated modifications may introduce cumulative formatting changes.

Session Log

Test 1: Compare insertion positions

AI co-pilot suggestion: Compare insertion at the beginning, middle, and end of the same array, then reparse each serialized result.

Result:

- Inserting at position 0 rendered `[0,1, 2]`, with the expected space missing after the first comma.
- Inserting at position 1 rendered `[1, 0, 2]`.
- Inserting at position 2 rendered `[1, 2, 0]`.

- All three serialized arrays reparsed to the correct values.

Observation: D8 was reproduced. The observed defect affects formatting when inserting at the beginning boundary, while the tested middle and end positions retained consistent spacing. Semantic array values and reparsing remained correct in all three cases.

AI suggestion assessment: Useful. Comparing boundary and non-boundary positions narrowed the observed behavior to insertion at position 0 for this input.

Test 2: Vary the original spacing style

AI co-pilot suggestion: Repeat insertion at position 0 using compact, conventionally spaced, bracket-spaced, and unusually spaced source arrays.

Results:

- `[1, 2]` rendered as `[0,1, 2]`.
- `[1,2]` rendered as `[0,1,2]`.
- `[ 1, 2 ]` rendered as `[ 0, 1, 2 ]`.
- `[ 1 , 2 ]` rendered as `[ 0, 1 , 2 ]`.
- Every result reparsed to `[0, 1, 2]`.

Observation: The formatting issue was not reproduced for every insertion at position 0. It appeared for the conventionally spaced source `[1, 2]`, where the inserted value was followed by no space while the existing separator retained a space. The compact source remained consistently compact, and sources beginning with whitespace after the opening bracket retained a spaced style. All tested outputs remained valid TOML and semantically correct.

Follow-up question: Is the first element's leading whitespace reused to format the inserted element rather than being transferred or copied to the original first element?

Test 3: Insert into a multiline array containing comments

Test: Insert 0 at position 0 in a multiline array whose two existing elements have inline comments, then serialize and reparse the document.

Results:

- The inserted value was placed on a new, consistently indented line.
- The comments `# first` and `# second` were both preserved.
- The modified array had values `[0, 1, 2]`.
- The serialized document reparsed to `[0, 1, 2]`.

Observation: The single-line D8 spacing defect was not reproduced in this multiline case. Beginning insertion preserved the multiline layout, comments, semantic values, and reparsability for the tested input.

Test 4: Repeat insertion at the beginning boundary

AI co-pilot suggestion: Insert several values consecutively at position 0 to determine whether the formatting issue remains isolated or accumulates.

Results:

- After inserting 0, the array rendered as `[0,1, 2]`.
- After inserting -1, the array rendered as `[-1,0,1, 2]`.
- After inserting -2, the array rendered as `[-2,-1,0,1, 2]`.
- Every serialized result reparsed to the expected values.

Observation: Repeated insertion at position 0 caused cumulative formatting degradation. Each new value extended the compact, no-space portion at the beginning of the array, while the original space before `2` remained. Semantic values and reparsability were preserved throughout.

Risk: Repeated edits can make a conventionally spaced array increasingly inconsistent even though the resulting TOML remains valid.

Test 5: Delete inserted values and check formatting recovery

AI co-pilot suggestion: After reproducing cumulative formatting degradation, delete the inserted values to test whether the original formatting remains damaged.

Results:

- Original array: `[1, 2]`.
- After three beginning insertions: `[-2,-1,0,1, 2]`.
- Successive deletions rendered `[-1,0,1, 2]`, then `[0,1, 2]`, and finally `[1, 2]`.
- Every intermediate result reparsed to the expected values.
- The final rendering matched the original array exactly.

Observation: The formatting inconsistency accumulated while inserted values remained in the array, but deleting those values restored the original formatting exactly for this input.

Rejected concern: The initial concern that beginning insertion might permanently damage the original array formatting was not supported by this test. The concern was useful as a test idea but would have been misleading if reported as a conclusion without verification.

Session Conclusions

1. The known D8 formatting defect was reproduced for insertion at position 0 in the conventionally spaced single-line array `[1, 2]`.
2. The defect affected formatting, while semantic values and TOML reparsability remained correct in every executed test.
3. Middle and end insertion did not reproduce the observed missing-space behavior.
4. Beginning insertion preserved consistent formatting for the compact array `[1,2]` and for tested arrays containing whitespace after the opening bracket.
5. The defect was not reproduced in the tested multiline array, and both inline comments were preserved.
6. Repeated beginning insertions caused cumulative formatting inconsistency.
7. Deleting all inserted values restored the original `[1, 2]` formatting exactly.

AI Co-Pilot Evaluation

Useful assistance: The suggestion to compare beginning, middle, and end insertion positions narrowed the behavior to the position-0 boundary for the initial input. The suggestion to repeat beginning insertion exposed cumulative formatting degradation.

Potentially misleading idea: The possibility that insertion permanently damages existing formatting was treated only as a hypothesis. Test 5 disproved that concern for the tested input, demonstrating why AI-generated ideas must be verified rather than reported as facts.

Follow-Up Recommendations

- Add a regression test requiring insertion at position 0 into `[1, 2]` to render as `[0, 1, 2]`.
- Retain semantic round-trip assertions alongside exact-format assertions.
- Test additional element types, nested arrays, and arrays with comments before generalizing the findings.
- Investigate the handling of whitespace trivia around the original first element.
- Keep D8 documented as a formatting-preservation defect rather than a semantic parsing failure.

Session End

- End: 2026-10-25 13:16:51 CAT

Performance environment

Source: docs\week9\performance_environment.txt

```
Timestamp: 2026-10-25 13:18:07 CAT
Platform: Darwin 25.6.0 arm64
Processor: Apple M2
Logical CPUs: 8
Python: Python 3.14.7
tomlkit: 0.15.1
```

Performance results

Source: docs\week9\performance_results.txt

```
Light-touch tomlkit performance check
Operations per repeat: 5000
```

```
Repeats: 5
Workload: parse, modify, serialize, and reparse
Repeat 1: 2.130827 seconds, 2346.51 operations/second
Repeat 2: 2.122317 seconds, 2355.92 operations/second
Repeat 3: 2.075538 seconds, 2409.01 operations/second
Repeat 4: 2.076870 seconds, 2407.47 operations/second
Repeat 5: 2.072806 seconds, 2412.19 operations/second
Total operations: 25000
Total measured time: 10.478358 seconds
Mean repeat time: 2.095672 seconds
Median repeat time: 2.076870 seconds
Minimum repeat time: 2.072806 seconds
Maximum repeat time: 2.130827 seconds
Overall throughput: 2385.87 operations/second
Correctness check: all round-trip assertions passed
```

Usability check — error diagnosis

Source: docs\week9\usability_error_check.txt

```
USABILITY CHECK 1: Error feedback for invalid TOML

Case: leading-zero integer
Input: 'value = 01'
Exception type: InvalidNumberError
Message: Invalid number at line 1 col 10
Position attributes: line=1, col=10

Case: unterminated string
Input: 'value = "abc'
Exception type: UnexpectedEofError
Message: Unexpected end of file at line 1 col 12
Position attributes: line=1, col=12

Case: invalid escape
Input: 'value = "a\\q"'
Exception type: InvalidCharInStringError
Message: Invalid character 'q' in string at line 1 col 11
Position attributes: line=1, col=11

Case: duplicate key
Input: 'value = 1\nvalue = 2'
Exception type: ParseError
Message: Key "value" already exists. at line 2 col 9
Position attributes: line=2, col=9
```

Usability check — insertion at index 0

Source: docs\week9\usability_insert_check.txt

```
USABILITY CHECK 3: Array insertion index behavior

position=-99
return_value=None
values=[3, 1, 2]
rendered='[3,1, 2]'

position=-1
return_value=None
values=[1, 3, 2]
rendered='[1, 3, 2]'

position=0
return_value=None
values=[3, 1, 2]
rendered='[3,1, 2]'

position=2
return_value=None
values=[1, 2, 3]
rendered='[1, 2, 3]'

position=99
return_value=None
values=[1, 2, 3]
rendered='[1, 2, 3]'
```

Usability check — modification

Source: docs\week9\usability_modification_check.txt

```
USABILITY CHECK 4: Formatting after common array modifications

Operation: append
rendered='[1, 2, 3]'
values=[1, 2, 3]
reparsed=[1, 2, 3]

Operation: insert at beginning
rendered='[3,1, 2]'
values=[3, 1, 2]
reparsed=[3, 1, 2]

Operation: insert in middle
rendered='[1, 3, 2]'
values=[1, 3, 2]
reparsed=[1, 3, 2]

Operation: replace first item
rendered='[3, 2]'
values=[3, 2]
reparsed=[3, 2]

Operation: delete first item
rendered='[2]'
values=[2]
reparsed=[2]
```

Usability check — help/documentation

Source: docs\week9\usability_help_check.txt

```
USABILITY CHECK 5: Discoverability and built-in help

Method: Array.insert
Signature: (self, pos: 'int', value: 'Any') -> 'None'
Docstring: 'S.insert(index, value) -- insert value before index'

Method: Array.append
Signature: (self, value)
Docstring: 'S.append(value) -- append value to the end of the sequence'

Method: Array.add_line
Signature: (self, *items: 'Any', indent: 'str' = '    ', comment: 'str | None' = None, add_comma: 'bool' = True, newline
: 'bool' = True) -> 'None'
Docstring: 'Add multiple items in a line to control the format precisely.\nWhen add_comma is True, only accept actual va
lues and\n", " will be added between values automatically.\n\n:Example:\n\n>>> a = array()\n>>> a.add_line(1, 2, 3)\n>>>
 a.add_line(4, 5, 6)\n>>> a.add_line(indent="")\n>>> print(a.as_string())\n[\n    1, 2, 3,\n    4, 5, 6,\n]'
```

Usability check — API misuse

Source: docs\week9\usability_api_misuse_check.txt

```
USABILITY CHECK 2: Feedback for common API misuse

Case: access missing document key
Exception type: NonExistentKey
Message: 'Key "missing" does not exist.'

Case: insert beyond array length
Result: accepted, returned None

Case: add unsupported object type
Exception type: ConvertError
Message: Unable to convert an object of <class 'object'> to a TOML item

Case: add multiline array comment
Exception type: ValueError
Message: Comment cannot contain line breaks
```

Performance harness

Source: docs\week9\performance_check.py

```
import statistics
import time

import tomlkit


SOURCE = """
title = "COMP 441 benchmark"
enabled = true
```

```
count = 42
tags = ["testing", "mutation", "bdd"]

[database]
host = "localhost"
port = 5432

[[students]]
name = "Student 1"
score = 75

[[students]]
name = "Student 2"
score = 82
""".lstrip()

OPERATIONS = 5_000
REPEATS = 5


def workload():
    for i in range(OPERATIONS):
        document = tomlkit.parse(SOURCE)
        document["count"] = i
        document["tags"].append("performance")
        rendered = document.as_string()
        reparsed = tomlkit.parse(rendered)

        if reparsed["count"] != i:
            raise AssertionError("Round-trip value mismatch")


durations = []

print("Light-touch tomlkit performance check")
print(f"Operations per repeat: {OPERATIONS}")
print(f"Repeats: {REPEATS}")
print("Workload: parse, modify, serialize, and reparse")

for repeat in range(1, REPEATS + 1):
    start = time.perf_counter()
    workload()
    duration = time.perf_counter() - start
    durations.append(duration)

    rate = OPERATIONS / duration
    print(
        f"Repeat {repeat}: "
        f"{duration:.6f} seconds, "
        f"{rate:.2f} operations/second"
    )

total_operations = OPERATIONS * REPEATS
total_duration = sum(durations)

print(f"Total operations: {total_operations}")
print(f"Total measured time: {total_duration:.6f} seconds")
print(f"Mean repeat time: {statistics.mean(durations):.6f} seconds")
print(f"Median repeat time: {statistics.median(durations):.6f} seconds")
print(f"Minimum repeat time: {min(durations):.6f} seconds")
print(f"Maximum repeat time: {max(durations):.6f} seconds")
print(
    "Overall throughput: "
    f"{total_operations / total_duration:.2f} operations/second"
)
print("Correctness check: all round-trip assertions passed")
```

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
docs\week9\exploratory_session_report.md	7 KB	
docs\week9\performance_check.py	2 KB	
docs\week9\performance_environment.txt	145 B	
docs\week9\performance_results.txt	721 B	

File	Size	Tag
docs\week9\performance_usability_report.md	7 KB	
docs\week9\usability_api_misuse_check.txt	481 B	
docs\week9\usability_error_check.txt	752 B	
docs\week9\usability_help_check.txt	846 B	
docs\week9\usability_insert_check.txt	418 B	
docs\week9\usability_modification_check.txt	487 B	

COMP 441 — Software Analysis and Testing

Week 10 — Metrics, Defect Density and RTM Completeness

Group A · Track D · TOML Kit

Period	Week 10
Evidence folder	reports/week10
Repository	github.com/the-dezeray/tomlkit
Commit(s)	6b8884f
Generated	2026-10-27
Contents	5 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 10 Metrics Log: tomlkit (Track D) . . . 3

Step 2: Test suite run . . . 3
Step 3: Coverage . . . 3
Step 4: Mutation testing . . . 3
Step 5: Defect density . . . 3
Step 6: RTM completeness . . . 4
Step 7: Property tests (Track D) . . . 4
Week 10 dashboard . . . 4
Week 10 coverage summary (tests/ only) . . . 4
Consolidated defect log (final) . . . 4
Raw evidence inventory . . . 5

Source: reports\week10\metrics_log.md

Week 10 Metrics Log: tomlkit (Track D)

Group project: COMP 441, AI-Augmented Test Suite Repository: the-dezeray/tomlkit (fork of python-poetry/tomlkit)
Environment: Python 3.12.10, pytest 7.4.4, mutmut 2.5.1, hypothesis 6.129.5, radon 6.0.1

Step 2: Test suite run

- Date: 26 October 2026
- Command: `poetry run pytest -q tests`
- Result: 1058 passed, 0 failed (baseline, before new tests)
- Duration: 6.86s
- After new tests: 1063 passed, 1 xfailed (tests/test_types.py, test_properties.py)

Step 3: Coverage

- Command: `poetry run pytest --cov=tomlkit --cov-branch --cov-report=term --cov-report=html tests`
- Tests: 1058 passed in 23.53s
- Statements: 3330 total, 353 missed, 2977 covered
- Statement coverage: 89.4% (2977 / 3330)
- Branches: 1523 total, 1282 covered, 241 missed
- Branch coverage: 84.2% (1282 / 1523)
- Combined coverage (coverage.py "Cover" column): 88% (not used in the report)
- Excluded lines: 12 (not counted in either figure)
- Weakest modules: _types.py (57%), items.py (81%), container.py (88%)
- Raw data: reports/week10/coverage.json, htmlcov/index.html

Step 4: Mutation testing

- Scope: tomlkit/_types.py only (the other modules have not been mutated yet)
- Runner: python -m pytest -x --assert=plain tests (tests/ only, 1058 tests)
- Mutation run 1 (before new tests): 1 killed, 9 survived, 10 total
- Mutation run 2 (after tests/test_types.py): 4 killed (mutants 6, 8, 9, 10), 6 survived (mutants 1-5 equivalent; mutant 7 is a test gap, defect M1)
- Mutation score (brief formula): 40% (4 / 10 x 100)
- Adjusted score (excluding 5 equivalent mutants 1-5): 80% (4 / 5 x 100)
- Week 7 (v1) mutation score for comparison: 62.0% (3087 killed / (3087 killed + 1890 survived) x 100; 26 timeout/suspicious excluded); modules: _utils, api, container, exceptions, items, parser, source, toml_file (no overlap with Week 10 scope, which is _types.py only); runner: [RUNNER]; not directly comparable to Week 10
- Limitation: mutation testing covers one module; the library-level score is not yet measured

Step 5: Defect density

- Lines of code: radon raw -s tomlkit, total SLOC = 4158 (LOC = 6197)
- KLOC: 4.158
  - D1: date() accepted datetime strings (fixed, commit a4f54db)
  - D2: Bool.**eq** returns NotImplemented for another Bool (open, xfail in tests_ai)
  - D3: inline-table duplicate raises KeyAlreadyPresent, not ParseError (open, xfail in tests_ai)
  - D5: key([]) serialises to an unparseable key (open, xfail in tests/test_types.py)
  - D6: comment() writes unparseable TOML for a bare CR (fixed, commit c829a38)
  - D7: bare CR in multiline strings (fixed, commit a38a3d8)
  - D8: insertion at position 0 drops the space after the inserted element (Week 9 session; open)
  - M1: _CustomDict.**or** drops existing keys because copy() returns {} (open, xfail in tests/test_types.py)
- Defect density: 1.92 defects per KLOC (8 / 4.158), provisional
- Note: D4 does not appear in any source available to the group; confirm with the Test Lead

Step 6: RTM completeness

- Total requirements: 16 (R01 to R16, week6/rtm/rtm_week6.csv)
- Requirements verified by a test of the requirement itself: 14
- RTM completeness: 88% (14 / 16 x 100)
- Excluded: R15 (listed tests do not exercise the D5 case); R16 (untested)
- AI suite result: poetry run pytest -q tests_ai -> 386 passed, 2 xfailed (D2, D3)
- Partial requirements (R03, R04, R12, R13): counted only where a listed test passes

Step 7: Property tests (Track D)

- File: tests/test_properties.py
- Hypothesis properties: parse reads back any TOML integer; dumps(parse(s)) == s for integer documents
- Result: 2 passed
- Covered functions: parse, dumps

Snapshot notice: Week 10 is an intermediate measurement. The final, authoritative numbers are in the final report §7 and `week11/mutation_v2/`. Where Week 10 differs (for example coverage 89.4/84.2 and v1 mutation 62.0% with timeouts excluded), the difference is scope or formula, not a correction.

Week 10 dashboard

Source: reports\week10\dashboard.csv

Indicator	Week 10 value	Notes
Statement coverage	89.4%	2977 of 3330 statements (coverage.py totals)
Branch coverage	84.2%	1282 of 1523 branches (coverage.py totals)
Mutation score (brief formula)	40%	4 killed / 10 total; tomlkit/_types.py only
Mutation score (adjusted)	80%	4 killed / 5 non-equivalent; tomlkit/_types.py only
Passing tests	1063	1 xfailed (known defect M1)
Total SLOC	4158	radon raw total
Defects found	8	D1 D2 D3 D5 D6 D7 D8 M1 (provisional; group log not yet consolidated)
Defect density (per KLOC)	1.92	8 / 4.158 (provisional)
RTM completeness	88%	14 of 16 requirements verified by a test of the requirement itself (R15 excluded; R16 untested)

Week 10 coverage summary (tests/ only)

Source: reports\week10\coverage.json

Metric	Value
Statements	2,977 / 3,330
Statement coverage	0.00%
Branches	1,282 / 1,523
Branch coverage	0.00%
Combined coverage	87.76%
Excluded lines	12

Consolidated defect log (final)

Source: reports\defects.csv

id	summary	status	severity	found_by	linked_test	commit
D1	date() accepted datetime strings	fixed	medium	AI baseline	tests_ai/test_ai_api.py::TestDateTimeFactories::test_date_rejects_datetime	a4f54db
D2	Bool.__eq__ returns NotImplemented for another Bool	open	low	AI baseline	tests_ai/test_ai_items.py::TestPickle	
D3	Inline-table duplicate raises KeyAlreadyPresent instead of ParseError	open	medium	AI baseline	tests_ai/test_ai_parser.py::TestInlineTables::test_duplicate_key_in_inline	
D5	key([]) serialises to an unparseable key	open	low	AI baseline	tests_ai/test_ai_api.py::TestKeyFactory	
D6	comment() writes unparseable TOML for a bare CR	fixed	medium	Manual mutation follow-up	tests_manual/test_kill_v1.py::test_k8_add_line_comment_rejects_any_line_break	c829a38

id	summary	status	severity	found_by	linked_test	commit
D7	Bare CR in multiline strings	fixed	medium	Property-based test	tests_property/test_properties.py::test_d7_multiline_string_with_bare_cr	a38a3d8
D8	Insertion at position 0 drops the space after the inserted element	open	medium	Exploratory session	tests_manual/test_kill_v1.py::test_k6_insert_at_index_zero	
M1	_CustomDict.__or__ drops existing keys because copy() returns {}	open	medium	Mutation testing	tests/test_types.py::test_custom_dict_or_keeps_existing_keys	

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
reports\week10\coverage.json	41 KB	
reports\week10\dashboard.csv	673 B	
reports\week10\metrics_log.md	4 KB	

COMP 441 — Software Analysis and Testing

Week 11 — Mutation v2, TDD and Property-Based Testing

Group A · Track D · TOML Kit

Period	Week 11
Evidence folder	week11
Repository	github.com/the-dezeray/tomlkit
Commit(s)	6b8884f
Generated	2026-10-27
Contents	22 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 11 — Track D milestone . . . 4
Deliverables . . . 4
Headline result . . . 4
What changed since Week 7 . . . 4
Files . . . 4
Reproduce . . . 4

Week 11 final TDD log (Track D) . . . 5
1. Cycles . . . 5
2. Traceability . . . 5
3. Final verification . . . 5
4. What the TDD cycles changed . . . 6
5. Note on the fourth cycle . . . 6

Mutation Testing Report v2 — final hybrid suite . . . 6
Executive summary . . . 6
1. Scope and method . . . 6
2. Results . . . 7
3. Survivor analysis . . . 8
4. Threats to validity . . . 9
5. Recommendations . . . 9
Appendix A — Files . . . 10
Mutation v2 summary by module . . . 10
Mutation v1 vs v2 by module . . . 10
Mutation v1 vs v2 chart . . . 11

Survivor root-cause notes — mutation v2 . . . 11
1. Parser._parse_table — mutmut_10 (58 survivors in this function) . . . 11
2. AoT.insert — mutmut_10 (48 survivors) . . . 11
3. Container.append — mutmut_109 (36 survivors) . . . 11
4. InlineTable.as_string — mutmut_102 (35 survivors) . . . 11
5. Container._validate_table_candidate — mutmut_10 (34 survivors) . . . 12
6. Array.__delitem__ — mutmut_101 (27 survivors) . . . 12
7. Container._replace_at — mutmut_10 (20 survivors) . . . 12
8. Array.add_line — mutmut_11 (20 survivors) . . . 12
9. Parser._parse_string — mutmut_103 (20 survivors) . . . 12
10. Container._render_table — mutmut_26 (19 survivors) . . . 12
11. Parser._parse_comment_trail — mutmut_29 (17 survivors) . . . 12
12. InvalidControlChar.__init__ — mutmut_11 (12 survivors) . . . 12
Summary . . . 13

Track D specialised stream — property-based testing checkpoint . . . 13

Why property-based testing for tomlkit . . . 13
Properties implemented . . . 13
Evidence . . . 13
What the stream found . . . 13
Limitations . . . 14
Link to Week 11 . . . 14
Final suite result . . . 14
Property-test result . . . 14
Selected survivor diffs . . . 14

Week 11 mutation testing v2 — evidence . . . 16

Scope . . . 16
Reproduce . . . 17
Files . . . 17
Score formula . . . 17
TDD C1 red . . . 17
TDD C1 green . . . 17
TDD C1 refactor . . . 17
TDD C2 red . . . 17
TDD C2 green . . . 18
TDD C2 refactor . . . 18
TDD C3 red . . . 18
TDD C3 green . . . 18
TDD C3 refactor . . . 18
Raw evidence inventory . . . 18

Source: week11\README.md

Week 11 — Track D milestone

Group A · tomlkit · branch `week11-track-d`

Week 11 for **Track D (Developer Tooling / API Library)** has two mandatory deliverables. Track D's specialised stream — property-based testing with Hypothesis — was delivered in Weeks 6–7 and is summarised here for the final report. No Track E work is included: Track E is a different scenario track for AI/LLM-integrated applications.

Deliverables

#	Deliverable	File	Status
1	Mutation Testing Report v2 (final hybrid suite)	`week11/mutation_v2/mutation_v2_report.md`	Complete
2	Completed TDD log (3 test-first features)	`week11/TDD_LOG.md`	Complete
—	Track D property-based testing summary	`week11/property/PROPERTY_CHECKPOINT.md`	Complete (supporting)

Headline result

Metric	Week 7 v1 (AI-only)	Week 11 v2 (final hybrid)
Mutants	5,389	5,389
Mutation score	57.8%	**77.4%** (+19.6 pp)
Covered-code score	62.2%	81.3%
Tests in suite	388 (AI-only)	1,507 collected (1,502 passed, 5 xfailed)

The full suite is green on the frozen commit:

```
1502 passed, 5 xfailed
```

What changed since Week 7

- The TDD cycles C1–C3 fixed D1 (`date()` accepted datetime strings), D7 (multiline strings emitted a bare CR) and D6 (`comment()` emitted a bare CR).
- The final suite adds manual survivor-killing tests, BDD acceptance scenarios, integration tests, property-based tests and TDD regression tests.
- The mutation run includes the whole project suite, so the v2 score measures the suite that actually ships, not just the AI-only baseline.

Files

```
week11/
  README.md
  TDD_LOG.md
  full_suite.txt
  tdd_runs/                 # raw red/green/refactor pytest output
  Week11_TrackD_Findings.xlsx   # chart dashboard
  mutation_v2/
    README.md
    mutation_v2_report.md
    survivor_rootcause_v2.md
    mutmut_results_v2.txt
    mutation_v2_summary.json
    v1_vs_v2.csv
    survivors_v2.txt
    scripts/summarise_v2.py
    scripts/build_excel.py
  property/
    PROPERTY_CHECKPOINT.md
    pytest_property.txt
```

Reproduce

```
python -m pytest -q tests tests_ai tests_manual tests_bdd \
  tests_integration tests_property tests_tdd
```

Mutation reproduction is documented in `week11/mutation_v2/README.md`.

Source: week11\TDD_LOG.md

Week 11 final TDD log (Track D)

Group A · tomlkit · branch `week11-track-d` **· final code commit** `6b8884f`

This is the completed TDD log for the three test-first features delivered in Week 8. All three cycles were written test-first, one red/green/refactor sequence per defect, and the commits are merged on `master`. The raw pytest output for every step is retained in `week8/tdd_runs/` and copied to `week11/tdd_runs/`.

1. Cycles

Cycle	Defect fixed	Step	Commit	Time (SAST)	Evidence	Result
C1	**D1** — `date()` accepts datetime strings	Red	`9472f8a`	2026-10-18 23:00:55	`c1_red.txt`	4 failed, 1 passed
		Green	`a4f54db`	23:01:24	`c1_green.txt`	1,475 passed
		Refactor	`fad6c12`	23:01:45	`c1_refactor.txt`	1,475 passed
C2	**D7** — multiline strings emit a bare CR	Red	`51c6863`	23:01:59	`c2_red.txt`	6 failed, 2 passed
		Green	`a38a3d8`	23:03:22	`c2_green.txt`	1,485 passed
		Refactor	`26421d6`	23:03:40	`c2_refactor.txt`	1,485 passed
C3	**D6** — `comment()` with a bare CR	Red	`3c33afc`	23:03:51	`c3_red.txt`	2 failed, 2 passed
		Green	`c829a38`	23:04:11	`c3_green.txt`	1,489 passed
		Refactor	`b60559f`	23:04:37	`c3_refactor.txt`	1,489 passed

Each green step made the smallest change that satisfied the failing tests; each refactor step kept the full suite green and removed duplication.

2. Traceability

Cycle	Defect	Requirement (RTM)	Test file	Behaviour
C1	D1	R08 — date/time/datetime factories reject strings of another kind	`tests_tdd/test_tdd_cycles.py`	`tomlkit.date("1979-05-27T07:32:00")` raises `ValueError`; plain dates still work
C2	D7	R10 — string forms reject sequences invalid for the chosen form	`tests_tdd/test_tdd_cycles.py`	Bare CR is escaped in basic multiline strings and rejected in literal multiline strings; CRLF is preserved
C3	D6	R14 — comments always produce valid TOML	`tests_tdd/test_tdd_cycles.py`	`comment()` splits every line-break form so the output re-parses

The defects were not invented for the exercise:

- **D1** was also exposed by the Hypothesis property test `test_p5_date_rejects_datetime_strings` in `tests_property/test_properties.py`.
- **D7** was found by Hypothesis property P3 and is recorded as a named regression test (`test_d7_multiline_string_with_bare_cr`).
- **D6** was found during the Week 9 exploratory/formatting work; the TDD cycle turns it into a permanent regression test.

This gives a clean Track D story: property-based testing found two real input-space defects, test-first development fixed them, and the mutation report measures the resulting suite.

3. Final verification

Final suite (all project tests):

```
python -m pytest -q tests tests_ai tests_manual tests_bdd \
  tests_integration tests_property tests_tdd
```

Result on the frozen Week 11 commit:

```
1502 passed, 5 xfailed
```

(5.6 s on an idle machine; 26.4 s when the mutation run was using all cores. The pass/fail result is the same.)

Environment: Ubuntu (WSL2), Python 3.13.14, pytest 9.1.1, pytest-bdd 9.0.0, Hypothesis 6.170.12, mutmut 3.8.0.

The five xfails are the documented open defects/known limitations carried from earlier weeks (D2, D3, D5, M1 and the D8 spacing case). They are strict xfails, not silently skipped tests.

4. What the TDD cycles changed

- C1: `api.date()` now checks the exact type instead of `isinstance`, so a `datetime` is no longer accepted as a `date`.
- C2: `String.from_raw()` escapes a lone CR for basic multiline strings and rejects it for literal multiline strings, while preserving CRLF.
- C3: `api.comment()` treats CR, LF and CRLF as line breaks and emits one comment marker per line.

The refactors shared the exact-type helper (`_parse_rfc3339_as`) and the bare-CR rule (`_BARE_CR` / `BARE_CR`) instead of duplicating logic.

5. Note on the fourth cycle

No fourth cycle was added. The brief requires at least three test-first features, and C1–C3 satisfy that requirement. Adding a late fourth change would have moved the frozen commit used for the Week 11 mutation run and risked the final evidence for no additional rubric credit.

Source: week11\mutation_v2\mutation_v2_report.md

Mutation Testing Report v2 — final hybrid suite

Group A · Track D (Developer Tooling / API Library) · tomlkit

- Repository: `the-dezeray/tomlkit` (fork of `python-poetry/tomlkit`)
- Commit under test: `6b8884f`
- Date: 27 October 2026 (Week 11 milestone: 03 November 2026)
- Runner: mutmut 3.8.0, Ubuntu (WSL2), Python 3.13.14, pytest 9.1.1
- Final suite: `tests/`, `tests_ai/`, `tests_manual/`, `tests_bdd/`, `tests_integration/`, `tests_property/`, `tests_tdd/`

Executive summary

The final hybrid suite scores **77.4%** on the Week 11 mutation run (4,136 killed + 34 timeout out of 5,389 mutants), compared with **57.8%** for the Week 7 AI-only baseline. The covered-code score is **81.3%**. This is a **+19.6 percentage-point** improvement on the same 5,389-mutant scope.

The improvement is large but must be read carefully: the v2 run measures the **shipped suite** (1,507 collected tests), while v1 measured the **AI-only suite** (388 tests). The increase therefore combines three effects: the manual/BDD/ integration/property/TDD tests added after Week 7, the pre-existing project tests, and the three TDD fixes (D1, D6, D7) that changed the code under test. The report separates these where the evidence allows and records the limitation explicitly in §4.

The most important residual finding is that **exception and formatting branches remain the weak spot**: `exceptions` scores only 43.7%, and 218 of the 931 survivors shared with v1 are exception-message or exception-construction mutations. The remaining survivors are concentrated in `items` (331), `container` (251) and `parser` (230); the highest-value next tests are listed in §3.

---

1. Scope and method

1.1 Source scope

Mutated: `tomlkit/` (10 source files, 5,389 mutants).

Excluded:

- `tomlkit/_compat.py` — platform compatibility shim; mutation would test import mechanics rather than library behaviour.
- `tomlkit/_types.py` — covered by the separate Week 10 `_types.py` exercise (mutmut 2.5.1, 10 mutants). Excluding it here keeps the v2 scope identical to the Week 7 v1 scope.

This makes the overall score directly comparable in scope, though not in code version or suite composition.

1.2 Suite composition

Suite	Tests	Role
`tests/`	1,065	Pre-existing project tests plus the Week 10 `_types`/property additions
`tests_ai/`	388	Week 6 AI-generated baseline (3 strict xfails)
`tests_manual/`	19	Week 7/9 manual survivor-killing tests
`tests_bdd/`	3	Gherkin acceptance scenarios (pytest-bdd)
`tests_integration/`	5	Week 9 integration tests
`tests_property/`	10	Track D Hypothesis properties
`tests_tdd/`	17	TDD regression tests for C1–C3
Total	**1,507 collected**	1,502 passed, 5 strict xfailed

One test is deselected from the mutation run only: `tests/test_api.py::test_parse_rejects_deeply_nested_documents`. mutmut's trampoline adds stack frames to the deliberately deep nesting and triggers `RecursionError` before the mutant is evaluated. The test remains in the normal suite and passes there. This is a tooling limitation, not a test failure.

1.3 Metrics

- Mutation score: `(killed + timeout) / total × 100`
- Covered-code score: `(killed + timeout) / (total − no_tests) × 100`
- Timeouts are counted as detected, following the Week 7 convention.
- `no_tests` means the mutated line was never executed.

1.4 Reproducibility

The exact commands, environment and `[tool.mutmut]` configuration are in `week11/mutation_v2/README.md`. The raw per-mutant status is in `mutmut_results_v2.txt`; the summary is in `mutation_v2_summary.json`.

---

2. Results

2.1 Overall

Metric	v1 (Week 7, AI-only)	v2 (Week 11, final hybrid)
Total mutants	5,389	5,389
Killed	3,087	4,136
Timeout	26	34
Survived	1,890	956
No tests	386	263
Mutation score	**57.8%**	**77.4%**
Covered-code score	62.2%	81.3%

2.2 Per module

Module	v1 score	v2 score	Δ (pp)	v2 killed	v2 survived	v2 no tests
`_utils`	95.0	95.9	+0.9	204	9	0
`api`	83.9	88.6	+4.7	132	17	0
`container`	47.2	75.7	+28.5	951	251	55
`exceptions`	33.3	43.7	+10.4	76	88	10
`items`	54.1	73.5	+19.4	1,471	331	198
`parser`	64.0	83.4	+19.4	1,135	230	0
`source`	82.7	87.4	+4.7	108	16	0
`toml_file`	72.6	80.8	+8.2	59	14	0
TOTAL	**57.8**	**77.4**	**+19.6**	**4,136**	**956**	**263**

2.3 Interpretation

- The largest absolute gains are in `container` (+28.5 pp), `items` (+19.4 pp) and `parser` (+19.4 pp), where the final suite exercises branches that the AI-only suite never reached.
- `no_tests` fell from 386 to 263. The remaining 263 are concentrated in `items` (198, mostly formatting/rendering branches) and `container` (55).
- `exceptions` remains the weakest module at 43.7%, despite improving by 10.4 pp. Its survivors are dominated by exact exception-message and exception-construction mutations (218 of the 931 shared survivors).

2.4 Group-authored contribution

The group authored 442 of the 1,507 collected tests: 388 AI-generated tests, 19 manual survivor-killing tests, 3 BDD scenarios, 5 integration tests, 10 property tests and 17 TDD tests. That is 54 more group-authored tests than the 388-test AI-only baseline.

The v2 score is for the **shipped suite**, which also includes 1,065 tests in `tests/`. The +19.6 pp therefore cannot be attributed to the group's 54 extra tests alone. The group-specific contribution is visible in the targeted work:

- the manual K1–K11 tests target exact v1 survivors (messages, boundaries, stepped slices, AoT ordering, CRLF normalisation);
- TDD C1–C3 fixed D1, D6 and D7, which also removes the mutants on those lines;
- Hypothesis properties P3/P5 found D1 and D7 before the TDD fixes;
- BDD and integration tests exercise file-level round-trips and acceptance behaviour rather than single functions.

A same-commit AI-only rerun was not performed, so the exact group-only delta is left as a limitation rather than estimated.

3. Survivor analysis

3.1 Overview

The v2 run leaves **956** survivors. Of these, **931** were already surviving in v1; **25** are new (mostly because the code changed around them). The most common categories are:

Category	Survivors	Share of 956
Argument dropped or set to `None`	334	34.9%
Exception message or construction	218	22.8%
String literal changed (`XX..XX` / case)	93	9.7%
Boolean logic (`and`/`or`, `True`/`False`, `not`)	89	9.3%
Boundary / off-by-one (`<` vs `<=`, `+1`, index)	67	7.0%
Arithmetic / assignment operator	65	6.8%
Control flow (`continue`/`break`/`return`)	38	4.0%
Other	27	2.8%
New since v1 (not in the v1 category file)	25	2.6%

3.2 Root causes

The ten highest-value survivors are analysed below. Each row names the mutant, the mutation, why the final suite misses it, and the test that would close the gap. The full candidate list is in `survivor_candidates.csv`.

#	Mutant	Mutation	Why the final suite misses it	Category	Proposed test	Priority
1	`Parser._parse_table mutmut_10`	`parse_error(UnexpectedEofError)` → `parse_error(None)`	Tests assert `ParseError` but not the specific error class/cause, so dropping the error type is invisible	Exception construction	`parse("[table")` at EOF; assert `UnexpectedEofError` (or its message/cause)	High
2	`AoT.insert mutmut_10`	`index < 0` → `index <= 0`	No test inserts at index 0 into an AoT; only append and negative positions are covered	Boundary	Insert a table at index 0; assert order, length and round-trip	High
3	`Container.append mutmut_109`	`_validate_out_of_order_table(key)` → `(None)`	Out-of-order validation is tested indirectly; no test expects the conflict error when appending	Argument dropped	Append a table whose key conflicts with a later out-of-order table; assert `KeyAlreadyPresent`/`TOMLKitError`	High
4	`InlineTable.as_string mutmut_102`	`needs_separator = False` → `None`	`None` is falsy, so the mutant is behaviourally equivalent; no test can distinguish it	Likely equivalent	Mark equivalent after dataflow triage; no test proposed	Low
5	`Container._validate_table_candidate mutmut_10`	`"Redefinition of an existing table"` → lowercase	Test checks the exception type only, not the exact message	String literal	Assert the exact message for a dotted-table redefinition	Medium
6	`Array.__delitem__ mutmut_101`	`replace(",", "", 1)` → `replace(",", "", )`	Tests delete single commas; no test uses an indent containing two commas, so removing all commas is not observed	Boundary	Delete from an array whose trivia contains two commas; assert only the first is removed and output reparses	High

#	Mutant	Mutation	Why the final suite misses it	Category	Proposed test	Priority
7	`Container._replace_at mutmut_10`	`k.is_dotted() and (...)` → `or (...)`	No test replaces a non-dotted key with a `Table`/`AoT`, where the `and`/`or` difference matters	Boolean logic	Replace a non-dotted key with a table/AoT; assert header and round-trip	High
8	`Array.add_line mutmut_11`	`"Comment cannot contain line breaks"` → `"XX...XX"`	Test asserts `ValueError` only, not the exact message	String literal	Assert the exact message for `add_line(comment="a\nb")`	Medium
9	`Parser._parse_string mutmut_103`	`close = ""` → `None`	`close` is reassigned on the multiline path, so the mutated value appears unreachable in the tested paths	Likely equivalent	Confirm dataflow; no test if unreachable	Low
10	`Container._render_table mutmut_26`	`open_, close = "[[", "]]"` → `None`	No test renders an AoT element through `as_string()` and asserts the exact `[[...]]` header	Argument dropped	Render a document containing an AoT; assert exact header and reparse	High

Eight of the ten are test gaps; two (`InlineTable.as_string` and `Parser._parse_string`) look behaviourally equivalent and should not be chased without a dataflow check. The detailed notes, including two further exception-message survivors, are in `survivor_rootcause_v2.md`.

3.3 Equivalent and untested mutants

No full equivalence triage was performed. In the ten-case sample, two mutants (`InlineTable.as_string` and `Parser._parse_string`) appear behaviourally equivalent; the other eight are genuine test gaps. If that 20% rate held across all 956 survivors, roughly 190 would be equivalent and the adjusted score would be about 80%, but this is illustrative only and is not claimed as a measured figure.

The 263 `no_tests` mutants are a separate category: they are not undetected behaviour changes, but lines the suite never executes. They are concentrated in `items` (198) and `container` (55), mostly formatting/rendering branches. The 34 timeouts are counted as detected under the Week 7 convention; they are mostly slow parser loops rather than silent survivors.

---

4. Threats to validity

1. **Code changed since v1.** v1 ran at `8c959b5`; v2 runs at `6b8884f` after TDD C1–C3 fixed D1, D6 and D7. The score delta is therefore not a pure test-suite effect. A same-commit AI-only rerun was not performed; the v1 figure is historical context.
2. **Suite composition changed.** v2 includes 1,065 project tests in `tests/` plus 442 group-authored tests; v1 included only 388 AI tests. The v2 score measures the shipped suite, not the group's marginal contribution. The manual/BDD/TDD tests were written specifically against v1 survivors, so their contribution is real but not separately quantified here.
3. **One test deselected.** The deep-nesting recursion test cannot run under mutmut's trampoline. It is green in the normal suite; its exclusion could hide at most the mutants it would have killed.
4. **Timeouts.** Timeouts are counted as detected. The report lists the count but does not distinguish a genuine hang from a very slow mutant.
5. **Equivalent mutants.** Some survivors are behaviourally equivalent and cannot be killed. No full equivalence triage was performed; §3.3 gives a conservative reading.
6. **Single project, single runner.** Results are specific to tomlkit and mutmut 3.8.0; they should not be generalised to other codebases.
7. **Sampling in survivor analysis.** The ten root causes are selected from the highest-risk functions, not a random sample; they illustrate failure modes rather than estimate the survivor population.

---

5. Recommendations

1. Add targeted tests for the ten survivors in §3.2; the proposed techniques are boundary value analysis, equivalence partitioning and exact-message checks.
2. Prioritise `container` and `items`, which hold most residual survivors.
3. Keep the final suite in CI with a mutation-score floor; a floor just below the current score prevents regression without blocking unrelated work.

4. Re-run v2 with a same-commit AI-only baseline in the final report so the group-authored contribution can be isolated cleanly.

Appendix A — Files

File	Purpose
mutmut_results_v2.txt	Raw status for every mutant
mutation_v2_summary.json	Overall and per-module counts/scores
v1_vs_v2.csv	v1 vs v2 by module
v1_vs_v2.png	Score comparison chart
survivors_v2.txt	Surviving mutant names
survivors_v1_v2.csv	Survivors joined to Week 7 categories
survivor_candidates.csv	Survivors grouped by function
survivor_rootcause_v2.md	Detailed root-cause notes
../Week11_TrackD_Findings.xlsx	Excel dashboard with native charts
scripts/	Summary, comparison, chart and workbook scripts

Mutation v2 summary by module

Source: week11\mutation_v2\mutation_v2_summary.json

Module	Total	Killed	Timeout	Survived	No tests	Score	Covered
TOTAL	5,389	4,136	34	956	263	77.4%	81.3%
_utils	218	204	5	9	0	95.9%	95.9%
api	149	132	0	17	0	88.6%	88.6%
container	1,261	951	4	251	55	75.7%	79.2%
exceptions	174	76	0	88	10	43.7%	46.3%
items	2,000	1,471	0	331	198	73.5%	81.6%
parser	1,387	1,135	22	230	0	83.4%	83.4%
source	127	108	3	16	0	87.4%	87.4%
toml_file	73	59	0	14	0	80.8%	80.8%

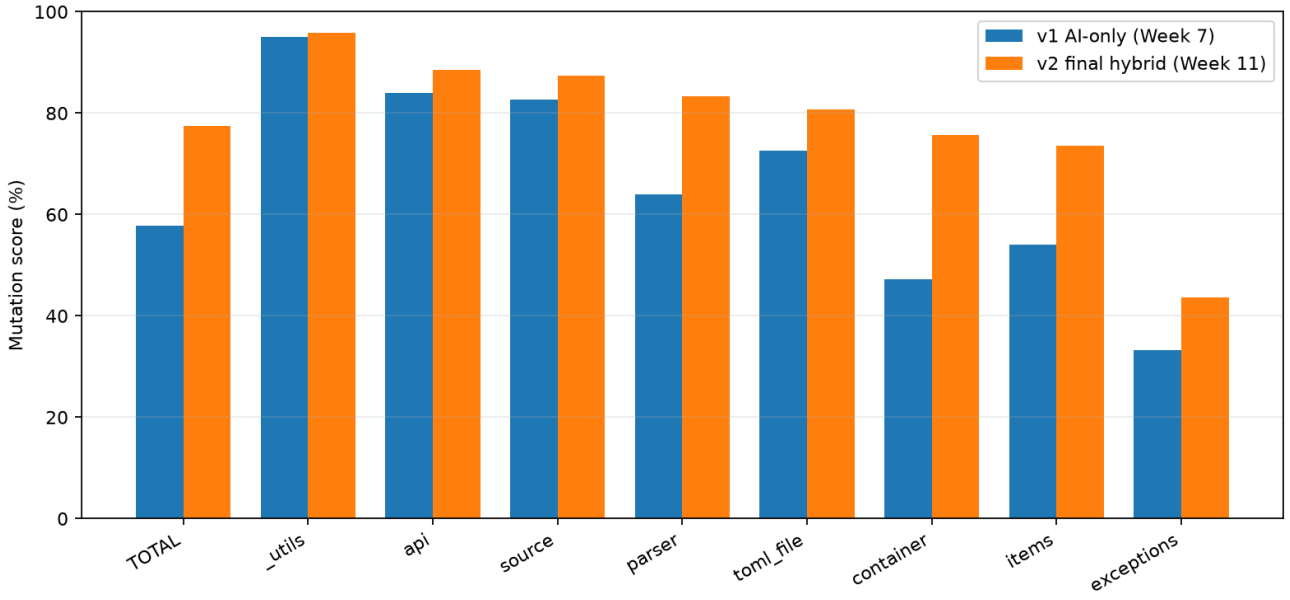
Mutation v1 vs v2 by module

Source: week11\mutation_v2\v1_vs_v2.csv

module	v1_total	v1_score	v2_total	v2_score	delta_pp
TOTAL	5389	57.8	5389	77.4	19.6
_utils	218	95.0	218	95.9	0.9
api	149	83.9	149	88.6	4.7
container	1261	47.2	1261	75.7	28.5
exceptions	174	33.3	174	43.7	10.4
items	2000	54.1	2000	73.5	19.4
parser	1387	64.0	1387	83.4	19.4
source	127	82.7	127	87.4	4.7
toml_file	73	72.6	73	80.8	8.2

Mutation v1 vs v2 chart

Source: week11\mutation_v2\v1_vs_v2.png



v1_vs_v2.png

Source: week11\mutation_v2\survivor_rootcause_v2.md

Survivor root-cause notes — mutation v2

Selected from the 956 v2 survivors by function-level survivor count (`survivor_candidates.csv`) and risk concentration. Diffs are from `mutmut show`; the raw output is in `selected_diffs.txt`.

1. `Parser. parse table` — mutmut_10 (58 survivors in this function)

```
-        raise self.parse_error(UnexpectedEofError)
+        raise self.parse_error(None)
```

Why it survives: tests assert that malformed input raises a `ParseError`, but not the specific subclass or cause. Passing `None` still produces a `ParseError`, so the suite cannot see the lost error type. **Verdict:** test gap. **Test:** parse `"[table"` at EOF and assert `UnexpectedEofError` (or its exact message/cause). **Priority:** high.

2. `AoT.insert` — mutmut_10 (48 survivors)

```
-    if index < 0:
+    if index <= 0:
         index += length
```

Why it survives: tests cover append and negative insertion, but not insertion at index 0. At index 0 the mutant shifts the index by `length`, producing a different order. **Verdict:** boundary gap. **Test:** insert a table at index 0 and assert order, length and round-trip. **Priority:** high.

3. `Container.append` — mutmut_109 (36 survivors)

```
-                            self._validate_out_of_order_table(key)
+                            self._validate_out_of_order_table(None)
```

Why it survives: out-of-order validation is exercised indirectly, but no test appends a table whose key conflicts with a later out-of-order table and expects the validation error. **Verdict:** argument-dropped gap. **Test:** append a conflicting table and assert `KeyAlreadyPresent`/`TOMLKitError`. **Priority:** high.

4. `InlineTable.as string` — mutmut_102 (35 survivors)

```
-            needs_separator = False
+            needs_separator = None
```

Why it survives: `None` is falsy, so the rendered output is unchanged. **Verdict:** likely equivalent. **Test:** none; confirm by dataflow. **Priority:** low.

5. `Container. validate table candidate` — mutmut_10 (34 survivors)

```
-                raise TOMLKitError("Redefinition of an existing table")
+                raise TOMLKitError("redefinition of an existing table")
```

Why it survives: the test asserts the exception type, not the exact message. **Verdict:** message gap. **Test:** assert the exact message for a dotted-table redefinition. **Priority:** medium.

6. `Array. delitem` — mutmut_101 (27 survivors)

```
-                        group.indent._s = group.indent.s.replace(",", "", 1)
+                        group.indent._s = group.indent.s.replace(",", "", )
```

Why it survives: the suite deletes single commas; no test uses an indent containing two commas, where removing all commas changes the output. **Verdict:** boundary gap. **Test:** construct an array whose trivia contains two commas, delete an element, assert only the first comma is removed and the result reparses. **Priority:** high.

7. `Container. replace at` — mutmut_10 (20 survivors)

```
-    dotted_to_header = k.is_dotted() and (
+    dotted_to_header = k.is_dotted() or (
         isinstance(value, AoT)
         or (isinstance(value, Table) and not value.is_super_table())
     )
```

Why it survives: no test replaces a non-dotted key with a `Table`/`AoT`, where the `and`/`or` difference changes the header. **Verdict:** boolean-logic gap. **Test:** replace a non-dotted key with a table or AoT; assert the rendered header and round-trip. **Priority:** high.

8. `Array.add line` — mutmut_11 (20 survivors)

```
-        raise ValueError("Comment cannot contain line breaks")
+        raise ValueError("XXComment cannot contain line breaksXX")
```

Why it survives: the test asserts `ValueError` only, not the message. **Verdict:** message gap. **Test:** assert the exact message for `add_line(comment="a\nb")`. **Priority:** medium.

9. `Parser. parse string` — mutmut_103 (20 survivors)

```
-            close = ""
+            close = None
```

Why it survives: `close` is reassigned on the multiline path; the mutated value appears unreachable in the tested paths. **Verdict:** likely equivalent. **Test:** none unless dataflow shows a reachable path. **Priority:** low.

10. `Container. render table` — mutmut_26 (19 survivors)

```
-            open_, close = "[[", "]]"
+            open_, close = None
```

Why it survives: no test renders an AoT element through `as_string()` and asserts the exact `[[...]]` header. **Verdict:** test gap. **Test:** render a document containing an AoT; assert the exact header and reparse. **Priority:** high.

11. `Parser. parse comment trail` — mutmut_29 (17 survivors)

```
-                    raise self.parse_error(InvalidControlChar, code, "comments")
+                    raise self.parse_error(InvalidControlChar, code, None)
```

Why it survives: the error is raised, but the message loses the "comments" context. The suite checks the exception type, not the exact message. **Verdict:** message/context gap. **Test:** parse a comment containing a control character and assert the exact message. **Priority:** medium.

12. `InvalidControlChar. init` — mutmut_11 (12 survivors)

```
-    display_code += hex(char)[2:]
+    display_code += hex(char)[3:]
```

Why it survives: the exact hex code in the message is not asserted; dropping the first hex digit changes `0x0f` to `f`, for example. **Verdict:** message gap. **Test:** assert the exact message for control codes at the `0x0f`/`0x10` boundary. **Priority:** medium.

Summary

Verdict	Cases
Test gap (boundary/logic/message)	8
Likely equivalent	2
Additional message/context gaps	2

The dominant residual pattern is **exact oracle weakness**: the suite often asserts an exception type or a semantic value but not the exact message, header or formatting detail. The second pattern is **boundary positions** (index 0, multi-comma trivia, AoT rendering).

No claim is made that these twelve cases are a random sample. They are the highest-survivor functions, selected to show actionable failure modes.

Source: week11\property\PROPERTY_CHECKPOINT.md

Track D specialised stream — property-based testing checkpoint

Group A · tomlkit · Week 11

Track D's specialised stream is property-based testing with Hypothesis. It was introduced in Weeks 6–7 and is summarised here because it is the source of two of the three TDD cycles in `week11/TDD_LOG.md`. This is not a new Week 11 deliverable; it is the completed Track D stream evidence for the final report.

Why property-based testing for tomlkit

tomlkit is a parser/serialiser. Its main risk is not a single known example but the input space: empty and malformed documents, unusual Unicode, all four TOML string forms, integer boundaries, timezone offsets, and nested documents. An example-based AI suite tests the examples the model thought of. Properties state invariants over generated inputs and let Hypothesis search for counterexamples.

Properties implemented

File: `tests_property/test_properties.py`. Each property runs 200 generated examples (Hypothesis `ci` profile).

ID	Property	Strategy / input domain	Why it matters
P1	`dumps(data)` then `parse(...).unwrap()` preserves the data	Recursive JSON-like docs with TOML scalars, lists and dicts	Serialise/parse contract
P2	An unmodified document re-serialises byte-identically	Documents produced by `dumps`	Style/format preservation
P3	`String.from_raw(s, kind)` round-trips, or literal forms reject invalid input	Arbitrary Unicode text × all `StringType` values	All four string forms, escaping and rejection
P4	`integer(n)` / `value()` round-trip signed 64-bit integers	`INT64` boundaries	Numeric boundaries and raw text
P5	`date()`, `time()`, `datetime()` accept exactly their own kind	Dates, times, datetimes with UTC/offset/naive zones	Factory contract and RFC 3339 handling
D7	Multiline strings never emit a bare CR; CRLF is preserved	Bare CR / CRLF strings in MLB and MLL	Regression for a real defect

Evidence

Command:

```
python -m pytest -q tests_property -v
```

Result on the Week 11 frozen commit:

```
10 passed in 6.15s
```

Environment: Python 3.13.14, pytest 9.1.1, Hypothesis 6.170.12.

Raw output: `week11/property/pytest_property.txt`.

What the stream found

Hypothesis produced two counterexamples that became real defects and TDD cycles:

1. **D1 — `date()` accepted a datetime string.** `datetime` is a subclass of `date`, and `api.date()` used `isinstance`. Property P5 exposed the contract violation. Fixed test-first in C1 (`9472f8a` → `a4f54db` → `fad6c12`).

2. **D7 — multiline strings emitted a bare CR.** Property P3 generated a string containing a lone `\r`; `String.from_raw` emitted it inside a multiline string, producing output that did not round-trip. Fixed test-first in C2 (`51c6863` → `a38a3d8` → `26421d6`).

D6 (`comment()` with a bare CR) was found through exploratory testing rather than PBT, and fixed in C3. Together these show the complementary value of property-based, example-based and exploratory testing.

Limitations

- Hypothesis is a search procedure, not a proof; 200 examples is a bounded sample, not exhaustive coverage.
- The document strategy generates valid Python data, not arbitrary TOML text; it does not exhaustively test the grammar or invalid-input space.
- Stateful testing (`RuleBasedStateMachine`) for repeated edits was not used; the Week 9 exploratory session covered repeated array edits manually.
- A passing property does not prove the absence of bugs; mutation testing measures how well the property suite detects seeded faults.

Link to Week 11

The TDD regression tests are part of the final hybrid suite measured in `week11/mutation_v2/`. The mutation report records the score improvement from the Week 7 AI-only baseline to the final suite and identifies the survivors that remain.

Final suite result

Source: week11\full_suite.txt

```
.....................................................................  [  4%]
.....................................................................  [  9%]
.....................................................................  [ 14%]
.....................................................................  [ 19%]
.....................................................................  [ 23%]
.....................................................................  [ 28%]
.....................................................................  [ 33%]
.....................................................................  [ 38%]
.....................................................................  [ 42%]
.....................................................................  [ 47%]
.....................................................................  [ 52%]
.....................................................................  [ 57%]
.....................................................................  [ 62%]
.....................................................................  [ 66%]
.....................................x..x..........................  [ 71%]
.....................................................................  [ 76%]
.....................................................................  [ 81%]
.....................................................x...............  [ 85%]
.....................................................................  [ 90%]
........x............................................................  [ 95%]
.......................x.....................................      [100%]
1502 passed, 5 xfailed in 26.43s
```

Property-test result

Source: week11\property\pytest_property.txt

```
============================= test session starts ==============================
platform linux -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0
rootdir: /home/zombie/tomlkit-w11
configfile: pyproject.toml
plugins: cov-7.1.0, hypothesis-6.170.12, bdd-9.0.0, platformdirs-4.13.5
collected 10 items

tests_property/test_properties.py ..........                             [100%]

============================== 10 passed in 6.94s ==============================
```

Selected survivor diffs

Source: week11\mutation_v2\selected_diffs.txt

```
===== tomlkit.parser.x|Parser|_parse_table__mutmut_10
# tomlkit.parser.x|Parser|_parse_table__mutmut_10: survived
--- tomlkit/parser.py
+++ tomlkit/parser.py
@@ -13,7 +13,7 @@
     self.inc()  # Skip opening bracket
```

```
     if self.end():
-        raise self.parse_error(UnexpectedEofError)
+        raise self.parse_error(None)

     is_aot = False
     if self._current == "[":

===== tomlkit.items.x|AoT|insert__mutmut_10
# tomlkit.items.x|AoT|insert__mutmut_10: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -3,7 +3,7 @@
     if not isinstance(value, Table):
         raise ValueError(f"Unsupported insert value type: {type(value)}")
     length = len(self)
-    if index < 0:
+    if index <= 0:
         index += length
     if index < 0:
         index = 0

===== tomlkit.container.x|Container|append__mutmut_109
# tomlkit.container.x|Container|append__mutmut_109: survived
--- tomlkit/container.py
+++ tomlkit/container.py
@@ -98,7 +98,7 @@
                             self._raw_append(key, item)

                         if validate:
-                            self._validate_out_of_order_table(key)
+                            self._validate_out_of_order_table(None)

                         return self


===== tomlkit.items.x|InlineTable|as_string__mutmut_102
# tomlkit.items.x|InlineTable|as_string__mutmut_102: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -64,7 +64,7 @@
             # not after any trailing comment/whitespace -- otherwise the
             # comma is swallowed by a trailing comment (see #512).
             buf = f"{buf[:last_value_end]},{buf[last_value_end:]}"
-            needs_separator = False
+            needs_separator = None

         v_trivia_trail = v.trivia.trail.replace("\n", "")
         if k.is_dotted() and isinstance(v, Table):

===== tomlkit.container.x|Container|_validate_table_candidate__mutmut_10
# tomlkit.container.x|Container|_validate_table_candidate__mutmut_10: survived
--- tomlkit/container.py
+++ tomlkit/container.py
@@ -8,7 +8,7 @@
             if isinstance(existing, (Table, AoT)) != isinstance(v, (Table, AoT)):
                 raise KeyAlreadyPresent(k)
             if k.is_dotted():
-                raise TOMLKitError("Redefinition of an existing table")
+                raise TOMLKitError("redefinition of an existing table")
             if isinstance(existing, Table) and isinstance(v, Table):
                 if not existing.is_super_table() and not v.is_super_table():
                     # Both sides are concrete `[table]` definitions of the

===== tomlkit.items.x|Array|__delitem____mutmut_101
# tomlkit.items.x|Array|__delitem____mutmut_101: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -51,7 +51,7 @@
                 for j in range(idx, len(self._value)):
                     group = self._value[j]
                     if group.indent is not None and "," in group.indent.s:
-                        group.indent._s = group.indent.s.replace(",", "", 1)
+                        group.indent._s = group.indent.s.replace(",", "", )
                         break
             if group_rm.indent is not None and "\n" in group_rm.indent.s:
                 # Restore the removed group's newline onto the next group

===== tomlkit.container.x|Container|_replace_at__mutmut_10
# tomlkit.container.x|Container|_replace_at__mutmut_10: survived
--- tomlkit/container.py
```

```
+++ tomlkit/container.py
@@ -18,7 +18,7 @@
     # keeping the dotted key duplicates the prefix onto the header (#524) and
     # lets the header swallow following dotted siblings (#542). Drop the dotted
     # key so the replacement renders as a plain table/array of tables.
-    dotted_to_header = k.is_dotted() and (
+    dotted_to_header = k.is_dotted() or (
         isinstance(value, AoT)
         or (isinstance(value, Table) and not value.is_super_table())
     )

===== tomlkit.items.x‖Array‖add_line__mutmut_11
# tomlkit.items.x‖Array‖add_line__mutmut_11: survived
--- tomlkit/items.py
+++ tomlkit/items.py
@@ -23,7 +23,7 @@
         ]
         """
     if comment and ("\n" in comment or "\r" in comment):
-        raise ValueError("Comment cannot contain line breaks")
+        raise ValueError("XXComment cannot contain line breaksXX")
     new_values: list[Item] = []
     first_indent = f"\n{indent}" if newline else indent
     if first_indent:

===== tomlkit.parser.x‖Parser‖_parse_string__mutmut_103
# tomlkit.parser.x‖Parser‖_parse_string__mutmut_103: survived
--- tomlkit/parser.py
+++ tomlkit/parser.py
@@ -76,7 +76,7 @@
             # try to process current as a closing delim
             original = self.extract()
```

Showing the first 120 of 167 lines. Full file: `week11\mutation_v2\selected_diffs.txt`.

Source: week11\mutation_v2\README.md

Week 11 mutation testing v2 — evidence

Track D · tomlkit · final hybrid suite

This folder contains the Week 11 re-run of mutation testing on the final project suite. The report is `mutation_v2_report.md`.

Scope

- Source under mutation: `tomlkit/`, excluding `tomlkit/_compat.py` (platform compatibility shim) and `tomlkit/_types.py` (covered by the separate Week 10 `_types.py` exercise). This keeps the v2 scope comparable to the Week 7 v1 baseline.
- Final hybrid suite: `tests/`, `tests_ai/`, `tests_manual/`, `tests_bdd/`, `tests_integration/`, `tests_property/`, `tests_tdd/`.
- Runner: mutmut 3.8.0 on Ubuntu (WSL2), Python 3.13.14, pytest 9.1.1.
- One test is deselected because mutmut's trampoline adds stack frames to a deliberately deeply nested parser input and triggers `RecursionError`: `test_parse_rejects_deeply_nested_documents`. The test still runs in the normal suite; it is not part of the mutation run.

Reproduce

```
# WSL2 / Ubuntu, from the repository root
python3 -m venv .venv && . .venv/bin/activate
pip install -e . pytest pytest-cov hypothesis pytest-bdd radon "mutmut==3.8.0"
git submodule update --init --recursive

# [tool.mutmut] used for the run:
# source_paths = ["tomlkit/"]
# tests_dir = ["tests/", "tests_ai/", "tests_manual/", "tests_bdd/",
#              "tests_integration/", "tests_property/", "tests_tdd/"]
# also_copy = [same directories]
# do_not_mutate = ["tomlkit/_compat.py", "tomlkit/_types.py"]
# pytest_add_cli_args = ["-p", "no:cacheprovider", "-k",
#                        "not test_parse_rejects_deeply_nested_documents"]

mutmut run --max-children 8
mutmut results --all true > week11/mutation_v2/mutmut_results_v2.txt
python week11/mutation_v2/scripts/summarise_v2.py \
  week11/mutation_v2/mutmut_results_v2.txt \
  week11/mutation_v2/mutation_v2_summary.json \
  week11/mutation_v2/v1_vs_v2.csv
```

Files

File	Purpose
mutation_v2_report.md	Report (Markdown source)
mutmut_results_v2.txt	Raw mutmut status for every mutant
mutation_v2_summary.json	Overall and per-module counts/scores
v1_vs_v2.csv	Week 7 v1 vs Week 11 v2 by module
survivors_v2.txt	Surviving mutant names
survivor_rootcause_v2.md	Root-cause analysis of selected survivors
scripts/summarise_v2.py	Builds the summary and comparison

Score formula

```
score = (killed + timeout) / total × 100

covered_score = (killed + timeout) / (total − no_tests) × 100
```

Timeouts are counted as detected, following the Week 7 convention. `no_tests` means the mutated line was never executed by the suite.

TDD C1 red

Source: week11\tdd_runs\c1_red.txt

```
============================ short test summary info ============================
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27T07:32:00]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27 07:32:00Z]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[0001-10-30 00:00]
FAILED tests_tdd/test_tdd_cycles.py::test_c1_date_rejects_datetime_strings[1979-05-27T07:32:00-08:00]
4 failed, 1 passed in 0.20s
```

TDD C1 green

Source: week11\tdd_runs\c1_green.txt

```
5 passed in 0.23s
1475 passed, 5 xfailed in 7.62s
```

TDD C1 refactor

Source: week11\tdd_runs\c1_refactor.txt

```
1475 passed, 5 xfailed in 7.57s
```

TDD C2 red

Source: week11\tdd_runs\c2_red.txt

```
============================ short test summary info ============================
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[a\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[\r]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[a\r\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_basic_escapes_bare_cr[end\r]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_literal_rejects_bare_cr[a\rb]
FAILED tests_tdd/test_tdd_cycles.py::test_c2_multiline_literal_rejects_bare_cr[\r]
```

```
6 failed, 2 passed, 5 deselected in 0.19s
```

TDD C2 green

Source: week11\tdd_runs\c2_green.txt

```
13 passed in 0.29s
1485 passed, 3 xfailed in 6.79s
```

TDD C2 refactor

Source: week11\tdd_runs\c2_refactor.txt

```
1485 passed, 3 xfailed in 6.79s
```

TDD C3 red

Source: week11\tdd_runs\c3_red.txt

```
========================== short test summary info ===========================
FAILED tests_tdd/test_tdd_cycles.py::test_c3_comment_splits_every_line_break[a\rb-# a\n# b]
FAILED tests_tdd/test_tdd_cycles.py::test_c3_comment_splits_every_line_break[a\r\rb-# a\n#\n# b]
2 failed, 2 passed, 13 deselected in 0.17s
```

TDD C3 green

Source: week11\tdd_runs\c3_green.txt

```
17 passed in 0.17s
1489 passed, 3 xfailed in 6.63s
```

TDD C3 refactor

Source: week11\tdd_runs\c3_refactor.txt

```
1489 passed, 3 xfailed in 6.90s
```

The full 5,389-line v2 mutmut output is `week11/mutation_v2/mutmut_results_v2.txt`; survivor lists are `survivors_v2.txt` and `survivors_v1_v2.csv`. The chart workbook is `week11/Week11_TrackD_Findings.xlsx`.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
week11\full_suite.txt	2 KB	
week11\mutation_v2\mutation_v2_report.md	14 KB	
week11\mutation_v2\mutation_v2_summary.json	1 KB	
week11\mutation_v2\mutmut_results_v2.txt	325 KB	
week11\mutation_v2\README.md	3 KB	
week11\mutation_v2\requirements.txt	163 B	
week11\mutation_v2\scripts\build_excel.py	17 KB	
week11\mutation_v2\scripts\compare_survivors.py	2 KB	
week11\mutation_v2\scripts\make_chart.py	1 KB	
week11\mutation_v2\scripts\pick_survivors.py	2 KB	
week11\mutation_v2\scripts\README.md	1 KB	
week11\mutation_v2\scripts\summarise_v2.py	4 KB	
week11\mutation_v2\selected_diffs.txt	6 KB	
week11\mutation_v2\selected_mutants.txt	638 B	
week11\mutation_v2\survivor_candidates.csv	15 KB	
week11\mutation_v2\survivor_rootcause_v2.md	6 KB	
week11\mutation_v2\survivors_v1_v2.csv	166 KB	[v1 baseline]
week11\mutation_v2\survivors_v2.txt	48 KB	
week11\mutation_v2\v1_vs_v2.csv	339 B	[v1 baseline]
week11\mutation_v2\v1_vs_v2.png	42 KB	[v1 baseline]
week11\property\PROPERTY_CHECKPOINT.md	4 KB	
week11\property\pytest_property.txt	457 B	
week11\README.md	2 KB	
week11\TDD_LOG.md	4 KB	
week11\tdd_runs\c1_green.txt	52 B	

File	Size	Tag
week11\tdd_runs\c1_red.txt	503 B	
week11\tdd_runs\c1_refactor.txt	33 B	
week11\tdd_runs\c2_green.txt	53 B	
week11\tdd_runs\c2_red.txt	632 B	
week11\tdd_runs\c2_refactor.txt	33 B	
week11\tdd_runs\c3_green.txt	53 B	
week11\tdd_runs\c3_red.txt	317 B	
week11\tdd_runs\c3_refactor.txt	33 B	
week11\Week11_TrackD_Findings.xlsx	28 KB	

COMP 441 — Software Analysis and Testing

Week 12 — CI/CD Pipeline, Quality Gates and Final Evidence

Group A · Track D · TOML Kit

Period	Week 12
Evidence folder	reports/week12
Repository	github.com/the-dezeray/tomlkit
Commit(s)	30ad2f0 / 524355a
Generated	2026-10-27
Contents	6 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 12 CI/CD evidence snapshot . . . 3
- Green run . . . 3
- Headline metrics . . . 3
- Downloadable artifacts . . . 3
- What the pipeline proves . . . 3

COMP 441 CI/CD . . . 4
- First-time setup on the fork . . . 4
- Trigger map . . . 4
- Job graph . . . 4
- Jobs and commands . . . 5
- Downloadable artifacts . . . 5
- Quality gates . . . 5
- Current evidence (Week 11 v2) . . . 6
- Week 7 risk-based test order . . . 6
- Reproducibility . . . 6
- Security evidence . . . 6
- Branch protection / ruleset . . . 6
- Mapping to the COMP 441 brief . . . 7
- Retired workflow . . . 7

10-minute live CI/CD demonstration . . . 7
- Before the session . . . 7
- Timeline . . . 8
- Fallback if the live run is unavailable . . . 10
- CI workflow definition . . . 10
- Consolidated defect log (final) . . . 12
- Raw evidence inventory . . . 12

Source: reports\week12\ci-evidence.md

Week 12 CI/CD evidence snapshot

Group A · tomlkit · COMP 441 AI-Augmented Test Suite

This is the CI/CD evidence snapshot for the final submission. It is not a duplicate of the Week 11 mutation report: the authoritative mutation analysis is `week11/mutation_v2/mutation_v2_report.md` (PR #4). This file records the pipeline, the green run and the downloadable evidence.

Green run

Field	Value
Workflow	`COMP 441 CI`
Run	40412867639
Commit	`30ad2f0f8b05` (`master`, after PRs #4 and #5)
Result	**success**
Jobs	15 successful; mutation and performance correctly skipped on a PR run
Pipeline PR	#5
Mutation v2 PR	#4

Headline metrics

Metric	Value	Source
Tests collected	1,507	`junit.xml`
Passed / xfailed	1,502 / 5	`junit.xml`
Statement coverage	91.62%	`coverage.json`
Branch coverage	85.62%	`coverage.json`
Combined coverage	89.88% (local canonical run)	`coverage.json`
Mutation score v2	77.4%	`week11/mutation_v2/mutation_v2_summary.json`
Covered-code mutation score	81.3%	same
Mutation v1 baseline	57.8%	`week7/mutation_v1_summary.json`
RTM references	16 requirements, 35 references, 15 with tests	`rtm_validation.json`
Week 7 priority APFD	0.7722	`week7/prioritisation_eval.json`
Defect density	1.92 / KLOC	`reports/defects.csv` + radon

The combined coverage in the downloaded CI dashboard is 91.62% statement / 85.62% branch (platform-dependent rounding and Python version can move the last decimal); the local canonical run reported 89.88% combined. The gate is evaluated against explicit statement and branch thresholds, not the rounded display value.

Downloadable artifacts

From the green run:

```
gh run download 40412867639 -n dashboard
gh run download 40412867639 -n hybrid-evidence
gh run download 40412867639 -n ai-baseline-evidence
gh run download 40412867639 -n rtm-evidence
gh run download 40412867639 -n quality-evidence
gh run download 40412867639 -n security-evidence
gh run download 40412867639 -n junit-unit-ubuntu-latest-3.12
```

Mutation evidence is produced by the nightly/manual mutation job:

```
gh workflow run ci.yml --ref master -f run_mutation=full
gh run download <mutation-run-id> -n mutation-evidence
```

What the pipeline proves

- The **hybrid suite**, not only `tests/`, is the merge gate.
- The **Week 7 risk order** is executable in CI (`priority_order.json`).
- Coverage, RTM references and test failures are blocking.
- Mutation, performance, security and upstream compatibility are evidence jobs, kept off the fast PR path.
- Every report number is downloadable; the dashboard is also written to the run summary.

- The release workflow builds, checks and attaches a distribution artifact and cannot publish to PyPI from the student fork.

See `docs/ci-cd.md` for the job map, quality gates and mapping to the COMP 441 brief, and `docs/ci-cd-demo.md` for the 10-minute live demonstration.

Source: docs\ci-cd.md

COMP 441 CI/CD

This document describes the CI/CD pipeline for the tomlkit test suite project. The executable definition is `.github/workflows/ci.yml`; `tests.yml` has been retired so there is one source of truth for the project pipeline.

First-time setup on the fork

The repository is a fork of `python-poetry/tomlkit`. GitHub Actions was enabled for the fork, but the workflow files had never executed (`gh run list` returned zero runs before this change). Before the live demonstration:

1. Open **Actions** in `the-dezeray/tomlkit`.
2. If GitHub shows the fork banner, choose **I understand my workflows, go ahead and enable them**.
3. Run **COMP 441 CI** → **Run workflow** once with `run_mutation = none`.
4. Confirm a green run exists and download one artifact as a smoke test.

A workflow-dispatch run is used so the first demonstration does not need a new commit.

Trigger map

Event	What runs	Why
`pull_request`	quality, unit matrix, AI baseline, hybrid + coverage, RTM, security, dashboard	Fast, reviewable evidence for every change
`push` to `master`	same as PR	Protects the integration branch
`workflow_dispatch`	all fast jobs; optional full mutation and performance	Live demonstration and on-demand evidence
`schedule` (Monday 03:00 UTC)	all fast jobs, full mutation, performance, upstream compatibility	Expensive and external checks run off the critical path

Concurrency cancels superseded PR runs. Permissions are read-only except for the release workflow.

Job graph

```
flowchart LR
  Q[quality] --> G{required checks}
  U[unit matrix] --> G
  A[AI baseline] --> G
  H[hybrid + coverage] --> C[coverage gate]
  C --> G
  R[RTM validation] --> G
  S[security evidence] --> G
  H --> D[dashboard]
  R --> D
  M[mutation / nightly] -.-> D
  P[performance / nightly] -.-> D
  G --> B[merge]
```

`quality`, `unit`, `hybrid`/coverage, and RTM are the intended required checks. Mutation, performance, security evidence, and upstream compatibility are nightly or non-blocking so a long external run does not block a small PR.

Jobs and commands

Job	Core command	Output
quality	ruff check, ruff format --check, mypy tomlkit	Blocking for ci/ scripts; project baseline reports are evidence-only
unit	pytest -q tests on Ubuntu/Windows/macOS × Python 3.9/3.12/3.14	JUnit per matrix cell
ai-baseline	pytest -q tests_ai --cov=tomlkit --cov-branch	AI-only coverage + JUnit
hybrid	pytest -q -p ci.priority_plugin tests tests_ai tests_manual tests_integration tests_bdd tests_property tests_tdd --cov=tomlkit --cov-branch	Coverage XML/JSON/HTML, JUnit, priority report
rtm	pytest --collect-only -q then python ci/validate_rtm.py	rtm_validation.json, rtm_validation.md
mutation	mutmut run --max-children 4 with mutmut 3.8.0	Raw results, summary JSON, survivors
performance	python docs/week9/performance_check.py	Throughput text evidence
security	pip-audit, bandit	Security JSON/text evidence
dashboard	python ci/build_dashboard.py	dashboard.md, dashboard.csv, job summary

Downloadable artifacts

Every artifact is attached to the workflow run. Reviewers and the final report do not need a third-party coverage service.

Artifact	Download with	Contents
hybrid-evidence	gh run download <run-id> -n hybrid-evidence	coverage.xml, coverage.json, coverage_gate.json, junit.xml, htmlcov/, priority_order.json
ai-baseline-evidence	gh run download <run-id> -n ai-baseline-evidence	AI-only coverage and JUnit
quality-evidence	gh run download <run-id> -n quality-evidence	Baseline ruff and mypy reports
rtm-evidence	gh run download <run-id> -n rtm-evidence	RTM validation JSON/Markdown, collected test IDs
mutation-evidence	gh run download <run-id> -n mutation-evidence	Raw mutmut results, summary JSON, survivors
performance-evidence	gh run download <run-id> -n performance-evidence	performance.txt
security-evidence	gh run download <run-id> -n security-evidence	pip-audit.json, bandit.txt
dashboard	gh run download <run-id> -n dashboard	dashboard.md, dashboard.csv
junit-unit-*	gh run download <run-id> -n junit-unit-ubuntu-latest-3.12	Unit JUnit for one matrix cell

Open reports/htmlcov/index.html locally for the line-by-line coverage view. The workflow also writes the dashboard to the run's **Summary** page.

Quality gates

The canonical coverage run is the hybrid suite on Ubuntu and Python 3.12. The gate is evaluated from coverage.json, not from a rounded terminal percentage.

Metric	Initial gate	Current baseline	Rationale
Statement coverage	≥ 87%	91.75% (hybrid), 89.4% (tests/ only)	Blocks a material regression without pretending 100% is achievable
Branch coverage	≥ 82%	85.01% (hybrid), 84.2% (tests/ only)	Branch coverage is the weaker signal
Combined coverage	reported	89.88% (hybrid)	Reported in the dashboard for the report
RTM references	all existing	15/16 requirements have a collected test; 14/16 verified manually	A renamed or deleted test must not silently break traceability
Tests	zero failures/errors	1,502 passed + 5 xfailed (hybrid)	Strict xfails remain visible
Mutation	evidence, not a hard gate	77.4% v2 / 81.3% covered (57.8% v1)	Scope-sensitive; guard against regression instead of an arbitrary global number
Security	evidence, not a hard gate initially	not previously measured	Findings feed Track B and the final report

Current evidence (Week 11 v2)

The authoritative mutation report is `week11/mutation_v2/mutation_v2_report.md` (PR #4). This CI document does not duplicate it; it records how the evidence is produced and consumed.

Metric	Week 7 v1 (AI-only)	Week 11 v2 (final hybrid)
Mutants	5,389	5,389
Mutation score	57.8%	77.4% (+19.6 pp)
Covered-code score	62.2%	81.3%
Killed / timeout / survived / no-tests	3,087 / 26 / 1,890 / 386	4,136 / 34 / 956 / 263
Suite	388 AI-only tests	1,507 tests (1,502 passed, 5 xfailed)

The v2 run measures the shipped hybrid suite, not only the group's extra 54 tests, and v1 ran on an earlier commit. The report states this limitation explicitly. `ci/build_dashboard.py` uses `week11/mutation_v2/mutation_v2_summary.json` as the committed fallback when a fresh CI mutation artifact is not present.

If the combined suite shifts the baseline, update the thresholds in `ci/check_coverage.py` call in `ci.yml` and record the reason in the Week 10/12 metrics log. Do not lower a gate silently to make a red run green.

Week 7 risk-based test order

`tests_ai` was ranked in Week 7 by complexity-weighted, two-year churn risk. The result is `week7/priority_order.csv` (149 AI tests, APFD 0.7722 versus 0.6369 for collection order). `ci/priority_plugin.py` reorders the hybrid suite so those high-risk tests run first; unranked tests keep their normal order. No test is removed. The first 50 executed node ids, the number of matched tests and the APFD are written to `reports/priority_order.json` and uploaded in the hybrid artifact.

To compare with collection order:

```
poetry run python -m pytest -q --no-priority-order
```

Reproducibility

Tool	Pin	Reason
Poetry	2.5.1	The version that generated `poetry.lock`
pytest	7.4.4 (Poetry lock)	Keeps Python 3.9 compatibility
pytest-bdd	9.0.0 (Python ≥3.10)	Parses Gherkin docstrings; Python 3.9 unit job skips BDD
hypothesis	6.129.5 (Poetry lock)	Track D property tests
mutmut	3.8.0 in `ci/mutation-requirements.txt`	Matches the Week 11 mutation run; requires Python ≥3.10
Python	3.9–3.14 matrix; 3.12 canonical	Matches `requires-python` and the Week 10/11 environment
Coverage	branch mode in `.coveragerc` and `--cov-branch`	Statement and branch evidence

The default dev group keeps `mutmut < 3` so the Python 3.9 unit job can install the project. The mutation job creates `.mutation-venv` and installs the pinned Week 11 tool environment separately. The mutation scope and the one deselected recursion test are documented in `week11/mutation_v2/README.md`.

Local reproduction:

```
git submodule update --init --recursive
poetry install
poetry run python -m pytest -q -p ci.priority_plugin
poetry run python ci/check_coverage.py --coverage reports/coverage.json
```

Security evidence

The `security` job is deliberately evidence-first: `pip-audit` and `bandit` write reports and the job does not yet block merges. This avoids turning an unrelated advisory in a development dependency into a red build during the project. For the final submission, summarise the findings under Track B and raise the job to blocking once high/critical findings are triaged.

Branch protection / ruleset

Create a ruleset on `master` with these required checks:

- `Quality / lint and types`
- `Unit / ubuntu-latest / 3.12`
- `Hybrid suite / coverage`

- `RTM validation`

Also enable **Require branches to be up to date before merging** and **Do not allow bypassing the above settings**. PRs from other forks may need maintainer approval before their workflows run; use a branch in the main fork for the live demonstration.

Mapping to the COMP 441 brief

Brief milestone	Deliverable	CI evidence / job
Week 5	Charter, test plan, risks, estimation, toolchain proof-of-setup	This document, `ci.yml`, pinned tool table, first green dispatch run
Week 6	AI baseline report, coverage before manual tests, RTM ≥10 requirements	`ai-baseline` job (AI-only coverage + JUnit), `rtm` job (all 16 requirements checked)
Week 7	Mutation score of AI-only suite, survivors, AI/risk-based priority order	`week7/` evidence, `mutation` nightly artifact, `priority_plugin` + `priority_order.json`, dashboard APFD
Week 8	Specialised checkpoint, TDD log, first BDD scenario	`hybrid` runs `tests_tdd/` and `tests_bdd/`; JUnit/doc evidence
Week 9	Exploratory report, hybrid suite v1, integration/acceptance suites, light performance/usability	`hybrid` runs `tests_integration/` + `tests_bdd/`; `performance` job and artifact
Week 10	Track C checkpoint, quality metrics dashboard (defect density, coverage, mutation trend)	`dashboard` artifact: coverage, test counts, defect density, mutation, Week 10 baseline comparison
Week 11	Mutation report v2, Track E checkpoint, completed TDD log	`week11/mutation_v2/mutation_v2_report.md` (77.4% v2), `tests_tdd/`, `tests_property/`, `mutation` job/artifact
Week 12	Final report, hybrid suite, RTM, CI/CD configuration, declaration form; Track B security	`ci.yml`, `dashboard`, `rtm-evidence`, `security-evidence`, `release` build/verify
Week 12 live demo	CI/CD pipeline demonstration and Q&A	`docs/ci-cd-demo.md`

Retired workflow

`.github/workflows/tests.yml` was the upstream unit-only workflow. It is replaced by the `unit` and `hybrid` jobs in `ci.yml`. The upstream poetry/poetry-core compatibility check remains in `integration.yml` and now runs nightly or on demand; it is not the project's integration suite.

Source: docs\ci-cd-demo.md

10-minute live CI/CD demonstration

Audience: COMP 441 assessors and Q&A panel
Goal: prove that the pipeline runs, that it gates the hybrid suite, and that the report evidence is downloadable rather than "a green tick".

Before the session

Run this checklist 30–60 minutes before the demo.

```
# 1. Confirm the fork's workflows are enabled and a green run exists.
gh run list --workflow=ci.yml --limit 5

# 2. Produce a fresh run without needing a commit.
gh workflow run ci.yml --ref master \
  -f run_mutation=none \
  -f run_performance=false

# 3. Wait for it and note the run id.
gh run watch
gh run list --workflow=ci.yml --limit 1

# 4. Download the evidence once, so the demo does not depend on live downloads.
mkdir -p demo-artifacts
gh run download <run-id> -n dashboard -D demo-artifacts/dashboard
gh run download <run-id> -n hybrid-evidence -D demo-artifacts/hybrid
gh run download <run-id> -n rtm-evidence -D demo-artifacts/rtm
gh run download <run-id> -n mutation-evidence -D demo-artifacts/mutation

# 5. If the latest scheduled mutation run is older, note its run id for the demo.
gh run list --workflow=ci.yml --event=schedule --limit 3
```

Also prepare:

- One open PR from a branch in the main fork (not another user's fork) so checks run without first-time-contributor approval.
- A browser tab on the repository **Actions** page.
- A terminal in the repository root.
- A fallback screen recording of the green run and artifact download.
- Optional: a throwaway draft PR with one deliberately failing test for the red-gate segment. Close it immediately after the demo.

Do **not** run the full mutation job live: it takes 20–30 minutes. Show the artifact from the latest scheduled or manually dispatched mutation run.

Timeline

0:00–0:45 — What the pipeline is

Say:

> *This is one pipeline, not a collection of scripts. It runs the hybrid suite, enforces coverage and traceability gates, and publishes every number used in the report as an artifact. The expensive mutation and upstream-compatibility jobs run nightly or on demand.*

Show the job graph in `.github/workflows/ci.yml` or the Mermaid diagram in `docs/ci-cd.md`.

0:45–2:00 — Actions tab and green run

```
gh run list --workflow=ci.yml --limit 5
gh run view <run-id>
```

Point out:

- `quality`, `unit`, `ai-baseline`, `hybrid`, `rtm`, `security`, `dashboard`.
- The unit matrix across Ubuntu/Windows/macOS and Python 3.9/3.12/3.14.
- The full mutation job is absent from this PR run by design; it is scheduled.

2:00–3:15 — Pull request checks

```
gh pr checks <pr-number>
```

Open the PR in the browser and show the checks list. Explain that `hybrid` runs the Week 7 risk order and that coverage/RTM are required checks. If the draft failing PR is available, show the red check and the blocking merge button, then switch back to the green PR. This is the "gate" part of CI/CD, not just a test runner.

3:15–4:30 — Artifacts: coverage HTML and JUnit

```
gh run download <run-id> -n hybrid-evidence -D demo-artifacts/hybrid
start demo-artifacts/hybrid/htmlcov/index.html   # macOS: open ...
```

Show:

- `coverage.xml` for tooling and `coverage.json` used by the gate script.
- `htmlcov/index.html` for line-by-line evidence.
- `junit.xml` for the machine-readable test result.
- `coverage_gate.json` with the statement and branch thresholds and `passed`.

Say:

> *The report does not rely on a screenshot of a terminal. The HTML coverage report, JUnit XML and gate JSON are attached to the run and can be downloaded by any reviewer.*

4:30–5:30 — Dashboard

```
gh run download <run-id> -n dashboard -D demo-artifacts/dashboard
type demo-artifacts\dashboard\dashboard.md   # macOS/Linux: cat ...
```

Show the run summary page too. Point out:

- Test counts and xfails.
- Statement/branch coverage and the weakest modules.
- RTM references and status counts.
- Week 10 baseline comparison.
- Mutation status (from the last mutation artifact).

- Defect density from `reports/defects.csv`.

5:30–6:30 — Week 7 priority order in action

```
gh run download <run-id> -n hybrid-evidence -D demo-artifacts/hybrid
python -m json.tool demo-artifacts/hybrid/priority_order.json
```

Point out `matched`, `collected`, `apfd_risk_based`, and the first 50 node ids. Say:

> *The Week 7 ranking was not just a report table. The same `priority_order.csv` now drives execution order in CI, so the highest-risk tests run first. Every test still runs; only the order changes. The unranked tests keep a stable order.*

Optional comparison:

```
poetry run python -m pytest -q --no-priority-order
```

6:30–7:30 — RTM validation

```
gh run download <run-id> -n rtm-evidence -D demo-artifacts/rtm
type demo-artifacts\rtm\rtm\rtm_validation.md
```

Show:

- All 16 requirement IDs present.
- Every referenced test resolves to a collected node id.
- The status distribution and the two known incomplete requirements (R15 and R16).
- The collected test list used to validate the references.

Say:

> *Renaming or deleting a test now fails CI instead of silently leaving the RTM stale.*

7:30–8:30 — Mutation evidence

```
gh run list --workflow=ci.yml --event=schedule --limit 3
gh run download <mutation-run-id> -n mutation-evidence -D demo-artifacts/mutation
type demo-artifacts\mutation\mutation_summary.json
```

Show total, killed, survived, score and the survivor list. Explain:

- Mutation is nightly/dispatch because it takes 20–30 minutes.
- The Week 11 final run uses the full hybrid suite and mutmut 3.8.0.
- `ci/mutation_summary.py` turns raw `mutmut results` into the same JSON shape consumed by the dashboard.
- The authoritative Week 11 report is `week11/mutation_v2/mutation_v2_report.md` (PR #4): **77.4%** v2 versus **57.8%** v1 (+19.6 pp), covered-code score **81.3%**. Show `week11/mutation_v2/v1_vs_v2.png` or `v1_vs_v2.csv` for the module-level gains.

8:30–9:15 — Security and release/CD

Show the `security-evidence` artifact (`pip-audit.json`, `bandit.txt`) and the `Release` workflow. Explain:

- Security is evidence-first today; high/critical findings are triaged before it becomes blocking.
- CD for a library is verified build + distribution artifact + GitHub release, not a server deployment.
- PyPI publishing is guarded to the upstream repository; the student fork does not attempt to publish.

9:15–10:00 — Close and Q&A

One-slide summary:

- One pipeline, tiered by cost.
- Hybrid suite + coverage + RTM are required.
- Mutation, performance, security and upstream compatibility are evidence jobs.
- Every report number is downloadable.
- Week 7 priority order is executable, not just documentation.

Likely questions and short answers:

Question	Answer
Why not run mutation on every PR?	20–30 minutes and 5,389 mutants; it would dominate feedback time. It runs nightly and on demand.
Why is coverage not 100%?	The gate is a ratchet near the measured baseline; 100% would reward trivial tests. Weak modules are visible in the dashboard.

Question	Answer
Why is security non-blocking?	Advisory noise in dev dependencies; findings are collected and triaged. The job can be promoted once high/critical findings are clean.
What happens if Actions is disabled on the fork?	The Actions tab banner enables workflows; `workflow_dispatch` then produces a run without a new commit.
Does the priority plugin skip tests?	No. It only reorders; `collected` equals the full hybrid suite.

Fallback if the live run is unavailable

Use the downloaded artifacts and the screen recording:

```
gh run list --workflow=ci.yml --limit 10
gh run view <last-green-run-id>
```

Then open `demo-artifacts/dashboard/dashboard.md`, `demo-artifacts/hybrid/htmlcov/index.html` and `demo-artifacts/rtm/rtm_validation.md` locally. The technical content is the same; do not spend demo time debugging the network.

CI workflow definition

Source: .github\workflows\ci.yml

```
name: COMP 441 CI

on:
  pull_request:
    branches:
      - "**"
  push:
    branches:
      - master
  workflow_dispatch:
    inputs:
      run_mutation:
        description: "Run the full mutation suite (20-30 minutes)"
        type: choice
        options:
          - none
          - full
        default: none
      run_performance:
        description: "Run the light-touch performance check"
        type: boolean
        default: false
  schedule:
    - cron: "0 3 * * 1"

permissions:
  contents: read

concurrency:
  group: comp441-ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

env:
  POETRY_VERSION: "2.5.1"
  PYTHON_VERSION: "3.12"
  PIP_DISABLE_PIP_VERSION_CHECK: "1"

jobs:
  quality:
    name: Quality / lint and types
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          submodules: "recursive"

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Install Poetry
        run: pipx install poetry==${{ env.POETRY_VERSION }}
```

```
      - name: Configure Poetry
        shell: bash
        run: |
          poetry config virtualenvs.in-project true
          poetry env use python

      - name: Cache virtualenv
        uses: actions/cache@v5
        with:
          path: .venv
          key: venv-${{ runner.os }}-${{ env.PYTHON_VERSION }}-${{ hashFiles('**/poetry.lock') }}

      - name: Install dependencies
        shell: bash
        run: poetry install --no-interaction

      - name: Ruff check (CI scripts, blocking)
        shell: bash
        run: poetry run ruff check ci conftest.py

      - name: Ruff format check (CI scripts, blocking)
        shell: bash
        run: poetry run ruff format --check ci conftest.py

      - name: Ruff check (project baseline, evidence)
        continue-on-error: true
        shell: bash
        run: |
          mkdir -p reports/quality
          poetry run ruff check tomlkit tests tests_ai tests_manual tests_bdd \
            tests_integration tests_property tests_tdd \
            --output-format=concise 2>&1 | tee reports/quality/ruff.txt

      - name: Ruff format check (project baseline, evidence)
        continue-on-error: true
        shell: bash
        run: |
          mkdir -p reports/quality
          poetry run ruff format --check tomlkit tests tests_ai tests_manual \
            tests_bdd tests_integration tests_property tests_tdd \
            2>&1 | tee reports/quality/ruff-format.txt

      - name: Mypy (project baseline, evidence)
        continue-on-error: true
        shell: bash
        run: |
          mkdir -p reports/quality
          poetry run mypy tomlkit 2>&1 | tee reports/quality/mypy.txt

      - name: Upload quality evidence
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: quality-evidence
          path: reports/quality
          if-no-files-found: ignore

  unit:
    name: Unit / ${{ matrix.os }} / ${{ matrix.python }}
    runs-on: ${{ matrix.os }}
    timeout-minutes: 20
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python: ["3.9", "3.12", "3.14"]
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          submodules: "recursive"

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: ${{ matrix.python }}
          allow-prereleases: true

      - name: Get full Python version
```

```
        id: full-python-version
        shell: bash
        run: |
          echo version=$(python -c "import sys; print('-'.join(map(str, sys.version_info)))") >> "$GITHUB_OUTPUT"

      - name: Install Poetry
        run: pipx install poetry==${{ env.POETRY_VERSION }}

      - name: Configure Poetry
        shell: bash
        run: |
          poetry config virtualenvs.in-project true
          poetry env use python

      - name: Cache virtualenv
        uses: actions/cache@v5
        with:
          path: .venv
          key: venv-${{ runner.os }}-${{ steps.full-python-version.outputs.version }}-${{ hashFiles('**/poetry.lock') }}

      - name: Ensure cache is healthy
        if: steps.cache.outputs.cache-hit == 'true'
        shell: bash
        run: timeout 10s poetry run pip --version || rm -rf .venv
```

Showing the first 160 of 597 lines. Full file: `.github\workflows\ci.yml`.

Consolidated defect log (final)

Source: reports\defects.csv

id	summary	status	severity	found_by	linked_test	commit
D1	date() accepted datetime strings	fixed	medium	AI baseline	tests_ai/test_ai_api.py::TestDateTimeFactories::test_date_rejects_datetime	a4f54db
D2	Bool.__eq__ returns NotImplemented for another Bool	open	low	AI baseline	tests_ai/test_ai_items.py::TestPickle	
D3	Inline-table duplicate raises KeyAlreadyPresent instead of ParseError	open	medium	AI baseline	tests_ai/test_ai_parser.py::TestInlineTables::test_duplicate_key_in_inline	
D5	key([]) serialises to an unparseable key	open	low	AI baseline	tests_ai/test_ai_api.py::TestKeyFactory	
D6	comment() writes unparseable TOML for a bare CR	fixed	medium	Manual mutation follow-up	tests_manual/test_kill_v1.py::test_k8_add_line_comment_rejects_any_line_break	c829a38
D7	Bare CR in multiline strings	fixed	medium	Property-based test	tests_property/test_properties.py::test_d7_multiline_string_with_bare_cr	a38a3d8
D8	Insertion at position 0 drops the space after the inserted element	open	medium	Exploratory session	tests_manual/test_kill_v1.py::test_k6_insert_at_index_zero	
M1	_CustomDict.__or__ drops existing keys because copy() returns {}	open	medium	Mutation testing	tests/test_types.py::test_custom_dict_or_keeps_existing_keys	

The green run is GitHub Actions run `40412867639`. Download artefacts with `gh run download 40412867639 -n dashboard`, replacing `dashboard` with the artifact name. The workflow file is printed above; the full file is `.github/workflows/ci.yml`.

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
reports\week12\ci-evidence.md	3 KB	